

任务图覆盖不到的级联损坏:无冲突柔性作业车间中的时空预约闭包

QI YU, Southeast University, PR China

WEI LI*, Southeast University, PR China

考虑运输的柔性作业车间调度若把运输时间取为按位置索引的常数矩阵,这一约定默认车辆互不共用同一时空资源,行驶时间因此只由布局决定,而不是路由决策或他车占用的结果。如果建模考虑了车辆对走廊的争用,有限车队在走廊上以时间窗预约通行,一辆车的让行会推迟另一辆车的到达。此时排程是否可行,便须同时满足任务依赖与预约表上的无冲突约束。在同时用工序依赖刻画加工先后、又用时间窗预约记录走廊占用的柔性作业车间调度中,扰动后重调度范围的现成做法仍是受影响工序重调度(AOR):先找出因设备故障或插单而无法按原计划执行的工序,再把同一工件的后续工序、以及同一机器上排在其后的工序一并纳入重排范围。这一做法覆盖了机器故障、插单等本身就是工序事件的扰动,却覆盖不到另一类——车间走廊在某段时窗内不可通行,或走廊上的车辆通行变慢。这类扰动不改任何工序指派:没有任何工序因此无法执行,AOR找不到起点,也就不重排任何工序;但原计划中落在阻断时段内的走廊占用已与禁行冲突,这些走廊占用规划不再可行,车辆阻塞或等待还会沿让行关系传给后续车辆。本文把这一缺口称作**任务图覆盖不到的级联损坏**:不是检测漏检,而是以工序为节点、以工件先后和同机工序先后为边的依赖图(任务图)不含走廊占用之间的让行关系,因而无法刻画走廊占用的规划失效如何在车辆之间传播。一旦模型把加工先后与走廊占用分成两套对象,却仍只用按工序是否受影响来划定重调度范围的方法,这种错位就会出现。

本文主张:在无冲突路由设定下,扰动的影响边界必须定义在时空预约图上。我们提出时空预约影响闭包(STRC, Spatiotemporal Reservation Closure):以与扰动时窗重叠的预约为种子,沿等待阻塞、同车接续、同工件接续与机器链等依赖取传递闭包,得到可释放、可冻结的损坏范围。让行边与 alternative arc 同源,本文不主张它是新对象;新的是它的用法——不选择弧,而在既定排程之上取闭包,使影响集从算法过程的副产品变为由图结构唯一确定的对象。在闭包上做有界修复(改路,失败则扩大释放域)是把该对象用起来的手段;「局部重规划+外侧冻结」与「失败扩域」两者在多智能体路径规划与局部重调度中均已标准做法,本文不以之为创新。

实验取 5 算例 $\times$ 10 种子(50 个算例-种子对),在同一无冲突执行器与同挂钟协议下对照任务图边界臂、热启动全局重解臂,以及两档便宜的全局重算。走廊阻断上任务图影响域在 50/50 上为空而闭包非空且原排程不可行;同修复引擎下仅替换边界定义时,任务图边界 0/50 恢复可行而闭包 50/50;闭包的结构闭合性与闭包外侧字段的逐条不变均在 50/50 上成立。把闭包单独隔离出来——换用「不划边界、释放全部未来」的对照臂,与有界修复共用同一引擎与同一冻结判据,只差释放集——耗时与完工时间几乎不变(R_2 对它的 $C_{\max}$ 为赢/平/输 4/46/0),而有界修复少改约 7 个百分点的预约(逐格 31/18/1, $p = 8.8 \times 10^{-7}$)。这定位了闭包买到什么:毫秒级响应来自单遍改路,闭包兑现的是稳定性。修复耗时中位 3.2 ms;同一实现里最便宜的一次全局重算比一次有界修复还贵,唯一更快的是不划边界的全局右移,而它在完工时间与稳定性上都更差。相对受假设 A2 约束的热启动全局重解,有界修复仍低两至三个数量级;完工时间在全局 18 个预算点上仍劣,但差距已比「从 $t=0$ 重排」那一档小得多,改动比例两边同在 50%-66% 量级——稳定性须与「只差释放集」的对照并读,不能挂在这一臂上。把算例自 8 工件 4 机推至 20 工件 10 机,闭包占比不升反微降。不利读数如实报出:完工时间上有界修复在全局 18 个预算点上劣于热启动全局重解,不存在交叉点;闭包亦不最小,它覆盖决策时刻之后活预约的约 92%,而上述稳定性收益的幅度与闭包排除掉多少成正比,故在闭包

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

接近该上界的算例上近乎消失。本文因此不宣称在完工时间上优于全局搜索; 贡献在于指出并修正影响边界的划定对象, 并给出与显式预约表设定相匹配的局部恢复路径。

CCS Concepts: • **Theory of computation** → **Scheduling algorithms**; • **Applied computing** → *Industry and manufacturing*; • **Computing methodologies** → *Planning and scheduling*.

Additional Key Words and Phrases: 柔性作业车间调度, AGV 无冲突路由, 时空预约表, 影响闭包, 级联损坏, 局部重调度, 走廊阻断

ACM Reference Format:

Qi Yu and Wei Li. 2026. 任务图覆盖不到的级联损坏: 无冲突柔性作业车间中的时空预约闭包. 1, 1 (September 2026), 30 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 引言

1.1 背景与动机

考虑运输的柔性作业车间调度长期依赖一个便利假设: 工件在机器之间的移动时间是一张按位置索引的常数矩阵。该假设等价于默认车辆互不干扰——行驶时间是布局的固有属性, 而不是路由决策的结果。一旦有限车队共享稀疏走廊网络, 并以半开时间窗预约走廊资源, 上述默认便不再成立: 一辆车的让行会推迟另一辆车的到达, 进而推迟工序开工与后续预约。此时, 一份「可行排程」同时包含工序指派、排序与一张走廊预约表; 扰动不仅可以打坏任务, 也可以只打坏预约。

静态一体化排程的研究已经表明, 显式无冲突路由与柔性指派的耦合会改变指派优劣次序, 而把拥堵写进评价、在决策中查询冲突信息, 是构建高质量排程的关键路径 [10, 19, 22]。那些工作主要回答: 如何在信息完备时建出一份好的无冲突排程。车间运行中的问题不同: 在执行时刻 t_{now} 发生扰动之后, 损坏落在何处、改到何处才够——以及这件事能否在毫秒至百毫秒量级完成, 而不必每次都重新打开种群搜索。

1.2 任务图覆盖不到的缺口

在考虑运输的柔性作业车间动态调度这一支中, 影响域通常沿任务依赖图界定: 由直接失效的工序或智能体作种子, 再按工件后继、机器链或固定跳数 θ 扩张, 然后只对影响域内任务重优化。近期两项处理机器故障与插单的 AGV 工作即如此——恢复范围分别取「故障后尚未加工的工序」与「按扰动传播分类后的工序与配送任务」[21, 26]。这一传统可上溯至受影响工序重调度 (AOR) [1], 是本文 R1 对照臂的原型 (第 2.1 节)。这一边界在「扰动本身就是一个任务事件」时往往合理——例如机器故障使若干工序无法按原指派执行。然而在显式预约模型里, 存在一类扰动, 其作用对象是走廊时窗而非工序指派。典型例子是走廊阻断: 某走廊在 $[t_a, t_b)$ 不可通行。阻断不修改任何机器指派或工序顺序, 因此任务图上的直接受扰集合 $T_{\text{direct}} = \emptyset$; 若框架以「空影响域」推出「无需重调度」, 则会系统性漏报——原排程中落在该时窗的预约已经失效, 并可能通过等待阻塞关系迫使后续车辆改道或推迟, 形成预约表上的级联。

本文强调用语: 所谓「覆盖不到」, 指的是任务图这一表示天然不含预约级联通道, 而不是传感器漏检或算法偶然漏报。在常数运输矩阵模型里, 这类损坏甚至无法被写下——走廊不是资源, 预约表不存在。因此, 问题本身附着于无冲突路由与显式预约设定; 这不是应用上的窄化, 而是表述该缺口所必需的模型前提。

据此, 我们区分两类扰动 (后文形式化):

- **A 类** (触碰任务图): 机械臂故障、加工时间显著偏差、插单, 以及车辆故障——任务图种子非空; 此时任务图边界与预约闭包都能看见该扰动, 不是本文主检验对象。车辆故障归入 A 类的理由见第 3.3 节: 它虽不改工序指派, 却使该车承运的工序不可执行, 经车-任务映射在任务图上留下非空种子。

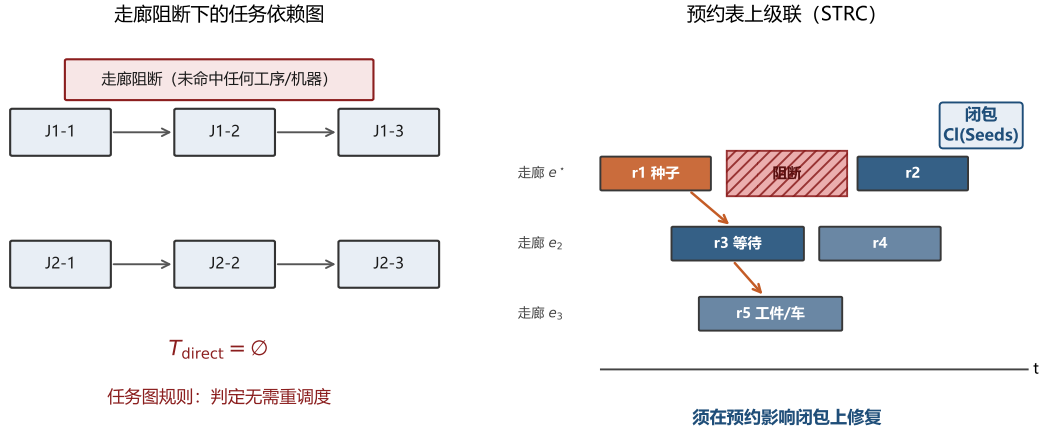


Fig. 1. 动机示意。左:走廊阻断不命中任何工序/机器,任务图直接集为空。右:同一扰动在预约表上产生种子,并沿阻塞与接续关系级联为时空预约闭包。

- **B 类**(只触碰预约表):走廊阻断或降速、局部通行权临时取消等——任务图种子为空,预约表种子与闭包非空。

B 类构成问题层的一击反例:不是「需要更好的修复启发式」,而是「在两张图并存的这一支模型里,可继承的影响边界在看不见该扰动的那张图上」。这个错位需要一个相当特殊的前提——模型必须同时持有任务图与让行图,却只用其中一张划界。缺少任一半都不会出现:铁路调度里闭塞区段就是机器,两图合一,区段阻断天然属于 A 类;多智能体路径规划里任务外生,没有任务图可供划错。上引两项 AGV 动态调度工作之所以未暴露该错位,也正是因为它们在模型里放掉了车辆冲突这一层(第 2.4 节),于是只剩一张图可划——这恰说明该错位是在两图并置时必然出现的,而非哪一篇的疏忽。故本文的主张写作「这一支」,而非对局部重调度文献的全称否定(第 2.5 节)。图 1 对照这一缺口:左侧任务图种子为空因而判「无需重调度」;右侧预约表上阻断窗命中种子并沿让行/接续边形成闭包。

1.3 主张与方法预告

本文主张三句话。第一,在无冲突柔性作业车间中,扰动影响边界应定义在时空预约图上,而非仅定义在任务依赖图上。第二,正确的影响对象是时空预约影响闭包 (STRC):以失效预约为种子,沿预约阻塞与调度接续关系取传递闭包;在给定边集上,闭包之外的预约不是种子的出边后继,从而「局部」对应一个可检验的对象,而不是启发式剪枝半径。第三,闭包上的有界修复——释放闭包内预约、外侧冻结、原车改路,失败则沿工件/车辆链扩大释放——是闭包的应用;A 类故障上的换车与改派已开口,改序仍不开放。这些动作都不是本文创新的挂靠点。

三句话共有一个立场,值得单独点明:影响闭包是一次测量,不是一个决策。本文不用阻塞关系去制导搜索——那样做买的是「改进一个决策」,其收益要经受关键链的稀释(消去一处走廊等待,往往只是把另一处约束顶成紧约束)。本文用同一份阻塞关系去划定损坏范围——买的是「改进一次测量」:给定边集上,一条预约是否落入闭包由传递闭包唯一确定,不随关键链上顶上来的是谁而变。这不等于主张闭包已含全部物理上必须释放的预约——边集是建模选择,闭合性是该选择下的定义推论(命题 2),最小性本文不作主张(注 2)。第 2.2 节用「测量不是决策」区分本文与铁路 alternative graph (那里选弧本身就是决策变量);第 1.5 节用同一句话说明本文与静态一体化工作对阻塞信息的不同用法。

与「每次扰动都热启动全局重解」相比,STRC 路径回答的是另一侧问题:在紧响应预算下能否快速恢复可行,并在结构上把修改限制在闭包内。外侧预约字段的逐条不变是给定边集与预提交成功时的充分条件(命题 3),不是无条件的恒等式;相对受 A2 约束的全局重解,有界修复这条路径响应更快,改动比例则已同量级。二者的归因不同,第 6.9 节把它们分开测过:响应之快来自「单遍改路、不重开搜索」这一机制形态,而改动之少须与「只差释放集」的对照并读。实验将表明二者的分工在响应时间一侧而不在解质量一侧——闭包修复以极短耗时恢复可行;全局重解在本文测到的每一个挂钟预算点上都取得更优完工时间——因而正确叙事不是「取代搜索」,而是「测量损坏范围,再决定用多便宜的协调就够」。

1.4 贡献

按问题层、框架层、机制层、方法层组织,本文贡献如下。

- (1) **问题层:指认任务图覆盖不到的级联损坏。** 在显式走廊预约下形式化 B 类扰动:按任务图直接集的常规定义,走廊阻断给出 $T_{\text{direct}} = \emptyset$;同时证明当预约种子非空时原排程仍不可行,从而「空影响域 $\Rightarrow$ 无需重调度」失效。该缺口既不在常数运输时间模型中可表述,也不在两图合一的模型(如铁路)中出现;它需要「两张图分立、却只用一张划界」这一特定层级。
- (2) **框架层:测量先于协调。** 动态恢复先由闭包给出释放集,再在闭包上做第 1 级改路。「局部重规划 + 外侧冻结」与失败扩域的机制形态非本文首创(第 2.5 节);框架层只主张释放集来自预约图上的对象,而不是任务图邻域或搜索过程的副产品。修复是第二步,不是标题主语。
- (3) **机制层:时空预约影响闭包(STRC)。** 定义种子、依赖边与传递闭包;证明结构闭合性(闭包外不是闭包内顶点的出边后继),并给出外侧字段不变的充分条件(预提交成功且不扩域时)。此处新的不是让行关系这一对象(见第 2.2 节),而是把影响集由算法过程的副产品改造成由图结构唯一确定的对象:它可陈述闭合性、可测量规模,且定义上不绑死某一修复启发式。同引擎消融表明可行与否来自边界定义;闭包相对平凡上界多买到的那部分,由「只差释放集」的对照臂单独归因(第 6.9 节)。
- (4) **方法层:闭包上的有界恢复与对照协议。** 实现第 1 级改路及失败扩域再修(两者的机制形态均非本文首创,见第 2.1、2.3 节);在统一无冲突执行器与同挂钟协议下,对照任务图边界、热启动全局重解,以及不划边界的便宜重算,报告可行性、耗时、预约变动与完工时间。扰动规模 φ 扫描用于表明响应时间不随扰动比例崩溃,不表示存在使有界修复完工时间更优的区间。
- (5) **边界维的覆盖矩阵,以及一条不利读数。** 在 50 个算例-种子对上,把四类扰动(走廊阻断、路段降速、车辆故障、机械臂故障)与两种边界定义交叉,给出任务图边界何时为空、闭包相对任务图释放集何时严格更大的边界读数;修复维四类都开口:B 类第 1 级均满,车辆故障 50/50,机械臂故障 50/50。同时如实报告:完工时间上有界修复对热启动全局重解全区间劣势、无交叉点;闭包不最小,覆盖 t_{now} 之后活预约的约 92%。

1.5 与静态一体化排程工作的关系

本文可独立阅读。若读者熟悉同一场景下的静态闭环双层排程工作,则二者关系可一句话概括:同一车间、同一预约表;前者用预约表构建排程,本文用预约表定位损坏。共享对象是显式路网、走廊独占与半开时窗预约;区别在时机与算法形态——静态工作在 $t = 0$ 做种群搜索以最小化完工时间,本文在 t_{now} 做单解有界恢复以恢复可行并把修改锚定在预约闭包上。

有一处表面上的不一致需要点破。若把同一套阻塞关系用来制导搜索(以「谁挡了谁」引导邻域),买的是「改进一个决策」,收益要经受关键链稀释——消去一处走廊等待,常常只是让另一条约束顶上来。本文用它去划定损坏范围,买的是「改进一次测量」:给定边集上闭包由传递闭包唯一确定,不随顶上来的是谁而变。二者并不矛

盾,期望回报本就不同。这仍不等于闭包已含全部物理上必须释放的预约(注 2)。影响闭包是一次测量,不是一个决策。第 3 节已把车间、预约表与两张图写自洽;未读过那些静态工作的读者可跳过本节。独立阅读时,上述静态工作视为相关文献即可。

1.6 篇章结构

第 2 节按「影响边界划在什么对象上」分四层回顾文献,并划清本文不主张什么。第 3 节给出车间与预约表设定、两张图、扰动二分与恢复问题;该节自洽,不依赖静态排程工作的读法。第 4 节定义 STRC 并陈述包含性(图 2、3)。第 5 节描述闭包上的有界修复与扩域策略(图 4)。第 6 节报告实验,并以案例甘特衔接统计与时间轴。第 7 节总结,并分节给出局限与后续工作。

2 相关工作

影响边界的划定并非新问题;真正区分各支文献的是边界划在什么对象上。据此本节分四层:① 任务图或时间轴(2.1);② 让行图,但与任务图合为一张(2.2);③ 只有让行图而无任务图(2.3);④ 两张图并存,却仍只用任务图划界(2.4)——最后一层是本文的位置。表 1 汇总。

2.1 任务图与时间轴上的影响边界

在任务依赖图上划界的原型是受影响工序重调度(Affected Operations Rescheduling, AOR)。Li 等人 [11] 以调度二叉树与 MRP 的净改变概念只修订「需要修订的工序」;Abumaizar 与 Svestka [1] 给出完整的 AOR 算法——只重排受扰动直接或间接影响的工序,其余保持原次序,并以效率与稳定性两个维度同完全重调度、右移重调度对比;Subramaniam 与 Raheja [18] 的 mAOR 把它从单一机器故障推广到插单、工时偏差、订单取消等多种扰动。本文的 R1 对照臂就是 AOR:以失效工序为种子,沿工件后继与机器链扩张,再只重排影响域内的部分。把它写成对照不是因为它弱,而是因为它是这一支的标准做法,且其种子定义直接决定了本文要检验的那件事。

沿时间轴划界是同样成熟的另一条路。match-up 调度 [3] 在扰动点之后选一个匹配时刻,只重排两者之间的部分,并给出该策略最优的条件;Aktürk 与 Görgülü [2] 用反馈机制确定匹配点。Local Rescheduling [9] 则把时间窗增量式扩张直到找到可行解,并明确指出 AOR 与 match-up 都绑定在生产特有的问题结构上。右移重调度则响应最快而不需要任何边界:它把未完成的部分整体后推,不判定谁受影响。近期一项考虑 AGV 与执行期故障的工作仍把「完全重调度 / 部分重调度 / 右移」并列为同一组取舍,并指出右移保住了稳定性却让出了优化空间 [21]。本文把它实现为对照臂之一(第 6.9 节),因为它正是「不划边界」这一端点。滚动时域与分解式方法在规模与质量之间折中 [25, 28];两篇综述系统梳理了完全重调度与修复式重调度的取舍 [16, 23]。

这一层给出两点纪律。其一,「应当划一个边界」不是本文的主张,它至少有三十年历史;本文能主张的只是边界该划在哪个图上。其二,「失败后扩大释放域」不是本文的机制创新——LRS 已是沿时间窗扩张,本文第 5.2 节沿依赖边扩张只是同一思路换了扩张方向。这一层的共同隐含前提是:扰动首先是一个任务事件。一旦扰动只使走廊时窗失效而不改指派,该前提不再成立,任务图种子可以为空,而排程已不可行。

2.2 让行图已有先例:alternative graph 与铁路实时重调度

「谁在排他资源上先于谁」这一关系被显式建成图,在铁路调度中已有二十余年历史。Mascis 与 Pacciarelli [14] 的 alternative graph 把析取图推广为:顶点是作业在闭塞区段上的占用,固定弧表达路径先后,成对的 alternative arc 表达「谁先通过该区段」。D'Ariano 等人 [5] 以该模型做实时列车重调度,在扰动后重算无冲突时刻表并最小化与原时刻表的偏离。本文的让行边与 alternative arc 是同一个对象,必须先承认这一点。两处区别决定了本文仍然成立:

- 用途不同:选弧 vs 取闭包。在 alternative graph 里,弧的选择本身是决策变量——求解即选一组弧使图无正环且目标最优。本文不选择弧:让行边由已经落盘的那份排程唯一确定,本文在这一既定选择之上取传递闭包,回答的是另一个问题——扰动之后哪些承诺必须作废。用一句话说,影响闭包是一次测量,不是一个决策。
- 模型层级不同,而这个差别正是本文问题的来源。在铁路模型里闭塞区段就是 机器,任务图与让行图合为一张;因此「区段阻断」在那里天然属于 A 类,不存在任务图看不见的问题。本文的缺口只在两张图分立的模型里存在。这既是对该文献的礼让,也是本文问题得以成立的前提。

2.3 无任务图的局部重规划:MAPF 的 Replan Affected

在多智能体路径规划中,「只为受影响者重规划、其余视作动态障碍」已是标准做法。Švancara 等人 [20] 在在线 MAPF 中明确给出 Replan All / Replan Single / **Replan Affected** 三档,并以在线独立性检测确定受影响集合; Malka 等人 [13] 在地图不准确的设定下同样「只为受观测变化影响的智能体重规划」。无冲突路径规划把时空占用作为一等约束,但任务集合与端点通常由外部给定 [27]。

因此「闭包内重规划 + 闭包外冻结」这一机制形态不是本文的创新,本文不作此声称。MAPF 之所以不会遇到本文的问题,是因为它根本没有任务图:任务外生,不存在「任务图看不见」这回事。它与本文的区别也不在机制,而在影响集的性质——独立性检测的输出是算法过程的副产品,随实现与终止条件而变;闭包是由图结构唯一确定的对象,可陈述闭合性、可测量规模,定义上不绑死某一修复启发式(第 4.2 节)。本文只在同一改路引擎上换过释放集,并未把闭包接到第二套修复引擎上实测。

2.4 两张图并存的场合:考虑运输的柔性作业车间

考虑运输车辆的柔性作业车间调度已有系统综述 [24]。近期组合优化工作仍普遍把行驶时间取为按位置索引的常数矩阵,并在此之上发展约束规划、元启发式与知识引导搜索 [6, 15, 17]。在该建模约定下,车辆互不占用同一时空资源,延误不是一车施加给另一车的量;走廊阻断这类事件甚至没有对应的模型对象。因此,「任务图覆盖不到的预约级联」在常数矩阵文献中既不会被提出,也无法被检验——这不是那些工作的缺陷,而是本文问题附着于另一建模层级的原因。

一旦下层改用显式走廊预约,两张图就同时存在了。制造场景中的拥堵感知派车以局部密度等代理量驱动选车,而不回头改机器指派 [8]。把柔性指派与无冲突运输真正耦合的工作相对较少:van Os 与 Basten [22] 形式化了带无冲突运输约束的柔性作业车间调度问题,并以分解方法给出可证最优基准;两阶段方案先固定排产再消解车辆冲突 [19];亦有以学习层排产、路径层消冲突并以有限反馈连接的框架 [10]。这些工作主要回答静态或准静态下如何构建可行且高质量的一体化排程。

把预约表与执行期扰动同时纳入的工作目前还没有,而两侧各自为什么没跨过来,文献里写在明处。持有预约表的一侧把执行期故障排除在模型之外:Sun 等 [19] 以时间窗 Dijkstra 消解车辆冲突,却在假设中声明车辆「始终可用,不考虑其故障与充电」,并在结尾把设备维护与故障列为后续研究。反过来,在柔性作业车间上真正处理执行期扰动的 AGV 工作,其影响边界一律落在任务对象上:Zhang 等 [26] 处理机器故障与插单,恢复范围是「故障之后尚未加工的工序整体重排」——一个纯粹的工序集合;Tang 等 [21] 按扰动传播把工序与配送任务分为保留、继续、重构三类,边界同样划在任务与配送任务上。而这两篇之所以能一致地这样划,恰恰因为它们不持有预约表:前者在模型假设中明写「忽略 AGV 之间的碰撞」。

于是缺口的性质不是「某篇工作用错了边界」,而是结构性的:工序级边界成立的前提正是没有预约表,而现有的预约表设定又恰好把执行期扰动排除在外,两个条件至今没有同时松开。一旦松开——而上述静态一体化工

表 1. 按「影响边界划在什么对象上」定位。最后一列是本文与该层的关系; 前三层本文均不主张新颖性。

层	代表工作	边界划在	有任务图?	与本文的关系
①任务图 / 时间轴	AOR [1, 11, 18]; match-up [2, 3]; LRS [9]	工序集合 / 时间窗	有	R1 即 AOR 。边界划定与失败扩域皆非本文首创
②让行图,两图合一	alternative graph [14]; 铁路实时重调度 [5]	闭塞区段占用	有,但与让行图同一张	让行边同源。区别:选弧(决策)与取闭包(测量);且区段即机器,故无本文缺口
③让行图,无任务图	Online MAPF 的 Replan Affected [13, 20]; LMAPF [27]	受影响智能体	无(任务外生)	「局部重规划 + 外侧冻结」为其标准做法,本文不作机制创新; 区别在影响集是过程还是对象
④两图并存	常数矩阵 FJSP-AGV [6, 15, 17, 24]; 一体化排程 [10, 19, 22]; 执行期扰动 [21, 26]	工序集合	有,且另有预约表	本文的位置 。两图并存与执行期扰动至今未同时出现; 一旦并置,继承自①的边界与损坏的载体错位

作都把执行期扰动列为下一步——可继承的现成边界只有第 2.1 节那一套,它在不改动任何工序指派的扰动会给出空的影响集(第 6.2 节实测 50/50)。本文要处理的就是这一格。

2.5 本文定位,以及本文不主张什么

综上,本文的主张必须收窄到第④层:在同时持有任务依赖图与显式走廊预约表的这一支调度文献中,可继承的影响边界仍是任务图那一套,而它对 B 类扰动系统性失效。写成对「局部重调度文献」的全称否定是不成立的——第②层不会犯这个错(两图合一),第③层不可能犯(没有任务图),缺口只在第④层这个特定的模型层级上存在。也不应写成「已有工作在这一格上划错了边界」:如上节所引,这一格里两图并存与执行期扰动至今没有同时出现过,因此本文指认的是一个在两者并置时必然出现的失效,而不是对某篇具体工作的纠错。相应地,R1 对照臂复现的是第①层的标准做法,不代表任何一篇第④层工作的实际选择。

相应地,本文不主张以下三件事,后文亦不再以创新的口吻提及: (i) 「只重规划受影响者、其余冻结为障碍」这一机制形态(第③层已有); (ii) 「单次修复失败后扩大释放域」(LRS 已有,只是沿时间窗而非沿依赖边); (iii) 「用让行关系组织冲突」这一建模对象(alternative graph 已有)。本文主张的是:该边界应改画在预约图上,并且这一改画可以被定义成一个有闭合性、可测量的对象,而不是又一条启发式规则。

3 问题描述

3.1 系统设定与记号

问题设定沿用带无冲突运输约束的柔性作业车间 [22]。本节把后文用到的对象一次写清;一份可行排程 S^0 视为给定输入,它在 $t = 0$ 如何被搜索出来不影响影响边界的定义。

3.1.1 车间与四组决策. 车间路由图 $G = (V, E)$ 强连通。机械臂占据固定节点 $\text{loc}(m)$; $v_0 \in V$ 为装卸站。 n 个工件、机器集合 M 、同构单载车队 $K = \{1, \dots, N_A\}$ 。工件 j 有固定工序链 $O_{j1} \rightarrow \dots \rightarrow O_{jn_j}$; 工序 O_{ji} 的候选机器集为 $\Omega_{ji} \subseteq M$, 在机器 m 上耗时 t_{jim}^P 。一份完整排程同时确定四组决策: (i) 每道工序的机器指派 x ; (ii) 各机器上的工序扫描序 π ; (iii) 各次搬运的车辆指派 y ; (iv) 每次车辆移动在 G 上的无冲突路径。记

$$S = (x, \pi, y, R),$$

其中预约表 R 是第 (iv) 组的显式记录,定义见下。

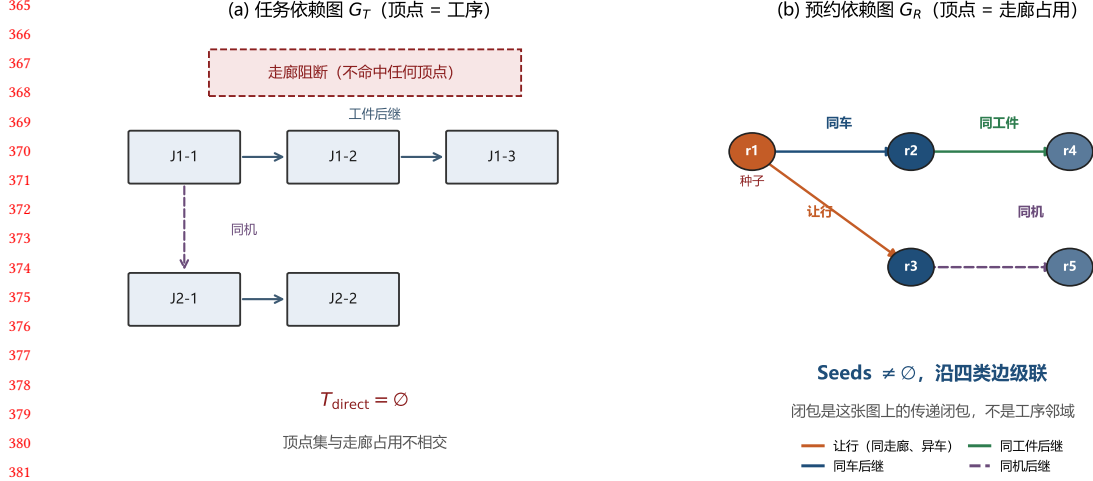


Fig. 2. 两张图的顶点集不相交。(a) 任务依赖图 G_T , 顶点是工序, 走廊阻断不命中任何顶点; (b) 预约依赖图 G_R , 顶点是走廊占用, 四类依赖边从种子向外级联。

运输任务由指派派生, 不是输入: 仅当相邻两道工序不在同一节点时才产生一次搬运。一次搬运拆成两段——空载取货与满载送达——各段有自己的任务标签 u (例如 O_{ji} 的空载段与满载段是两条记录)。后文的释放、冻结与改路都以段为粒度, 而不是整道工序。

3.1.2 走廊预约与无冲突. 每条走廊 $e \in E$ 是排他资源, 通行时间 τ_e 已知; 两个方向共享同一份占用, 因而任一时刻至多一辆车占用 e , 迎面冲突与同向超车一并排除。占用取半开区间: 第二辆车可以恰好在区间右端点进入。节点上设容量充裕的停靠位, 等待发生在节点上且总是被允许。一条预约记录为

$$r = (e, [t^s, t^e], k, u) \in R,$$

即车辆 k 在 $[t^s, t^e]$ 占用走廊 e , 所属运输段为 u 。同一走廊上任意两次不同占用必须满足 $[t^s, t^e] \cap [t^{s'}, t^{e'}] = \emptyset$ 。路径只写入 R 报告为空闲的时窗, 故无冲突是构造出来的, 不是事后修补的。

时间窗路由是后文改路与对照臂共用的原语: 给定起终点、最早出发时刻与当前 R , 它返回一条与已有占用无交的通路及各走廊进入时刻。实现上采用时间窗 Dijkstra [19]; 本文不改这条原语, 只改哪些运输段被送进路由器。与 van Os 预约节点、Lyu 另加节点容量不同 [12, 22], 本文预约的是边; 第 7.2 节交代由此带来的算例口径限制。

3.1.3 两张顶点不相交的图. 任务依赖图 $G_T = (T, E_T)$ 以工序为顶点, 边为工件先后与同机先后; 它不含走廊占用之间的阻塞关系。预约依赖图 $G_R = (R, E_R)$ 以 R 中的预约为顶点, 边为让行、同车、同工件与同机所诱导的依赖 (定义 4)。两张图的顶点集不相交: 一边是加工, 一边是通行。因此存在一类只改走廊时窗、不改任何工序参数的扰动—— G_T 上种子为空, 而 G_R 上种子非空, 且原排程已不可行。图 2 对照这一分立; 图 1 是它在走廊阻断下的后果。

3.2 执行期假设

执行期约定与物理设定如下, 以便把读数归因到边界定义而不是模型差异:

表 2. 本文常用记号

符号	含义
$G = (V, E), \tau_e$	走廊路网;走廊 e 的通行时间
$v_0, \text{loc}(m)$	装卸站;机械臂 m 所在节点
x, π, y	机器指派、扫描序、车辆指派
$R, r = (e, [t^s, t^e], k, u)$	预约表;一条半开占用(车辆 k , 段标签 u)
G_T / G_R	任务依赖图 / 预约依赖图
$\mathcal{S}^0, \mathcal{S}'$	扰动前排程、恢复后排程
$\mathcal{D}, t_{\text{now}}$	扰动事件、发生(决策)时刻
$T_{\text{direct}}, T_{\text{impact}}$	任务图直接/ θ -跳影响域
$\text{Seeds}(\mathcal{D}), U$	预约种子;释放集
$\text{Cl}(\cdot)$	时空预约影响闭包(STRC)
φ	受扰未来工序比例(规模扫描)

- A1. 初始排程可行。 $\mathcal{S}^0$ 在扰动前通过无冲突校验;预约表与工序时间表一致。
- A2. 已完成占用不回溯。 只重协调 $t_{\text{end}} > t_{\text{now}}$ 的未来预约与未完工工序; $t_{\text{end}} \leq t_{\text{now}}$ 的占用字段不可改写。
- A3. 单次、完全观测。 扰动在 t_{now} 被完整观测,且不与其它扰动叠加;扰动流、预测误差与滚动响应不在本文范围内。
- A4. 走廊排他、缓冲充裕、车队同构单载。 走廊仍为排他半开资源,阻断以附加占用 (或禁行窗)注入路由层。有限缓冲、充电、加减速不建模;全部车辆在计划开始时空载停于 v_0 。
- A5. 允许的恢复动作。 B 类扰动保持原机器指派 x 与扫描序 π ,只改路;失败时扩大释放集。A 类故障额外允许:车辆故障把故障车从车队摘除并换车;机械臂故障把未完工工序改派到其它可行机。扫描序 π 始终不变,不重开搜索。实验中全局重解对照臂可以改 x 与 π (第 5.3 节)。

3.3 扰动二分

定义 1 (A/B 类扰动). 若扰动直接使某些工序的指派或可执行性失效(机械臂故障、显著工时偏差、插单、车辆故障等),称其为 **A 类**;若扰动只使某些走廊时窗不可用、而不使任何工序不可执行(走廊阻断、路段降速等),称其为 **B 类**。

注 1 (车辆故障为何属于 A 类). 车辆故障不改动任何工序的机器指派,乍看应与走廊阻断同类。但定义 1 的判断是可执行性而非指派:一辆车抛锚后,它承运的那些工序在原计划下无法开工,故经「车辆→所承运工序」的映射,任务图上留有非空种子。这一归类是保守的,它把一类边缘情形判给了对本文不利的一侧——A 类是任务图边界能看见的那一类。需要指出,只有当重调度框架维护了车-任务映射时该种子才非空;若框架只跟踪工序与机器 (AOR 的常见实现即如此),车辆故障同样会漏报。本文不依赖这一点立论,第 6.5 节按维护了该映射的强口径报告 A 类读数。

定义 2 (任务图直接集与影响域). 对扰动 $\mathcal{D}$,令 $T_{\text{direct}}(\mathcal{D})$ 为被故障智能体或失效工序直接命中的任务 id 集合。对 B 类走廊扰动约定 $T_{\text{direct}} = \emptyset$ 。在 G_T 上从 T_{direct} 出发做深度至多 θ 的扩张,记为 $T_{\text{impact}}(\mathcal{D}, \theta)$ 。

定义 3 (预约种子). 种子集 $\text{Seeds}(\mathcal{D}) \subseteq R$ 按扰动类型给出,一律只取 $r.t^e > t_{\text{now}}$ 的预约。

- (a) **走廊阻断/降速** $\mathcal{D} = (e, [t_a, t_b]): \{r : r.e = e, [r.t^s, r.t^e] \cap [t_a, t_b] \neq \emptyset\}$ 。多微阻断时取各窗种子之并。两者共用这条种子规则(原排程按时长未缩放的 τ 写出,与窗重叠的占用都已失效),故边界维给出同一个闭包——第 6.5 节据此说明为何不把二者的边界读数算作两次独立验证;修复维二者已分开,降速只缩放与窗重叠的那一段进度,而不再写成整段禁行,也不再把有交的穿越整段乘倍率。

(b) 车辆故障 $\mathcal{D} = (k)$: 该车在 t_{now} 之后的全部预约 $\{r : r.k = k\}$ 。

(c) 机械臂故障 $\mathcal{D} = (m, F)$, F 为该机上尚未完工的工序: 对每个工件取 F 中最小工序号 $i_{\min}(j)$, 种子为各工件自该工序起(含)的全部未来预约。之所以沿工件链向后取而不只取该工序自身, 是因为进站运输可能已在 t_{now} 前结束, 若只取自身, 闭包会短于任务图影响域, 对照即不公平。

命题 1 (B 类上的任务图空洞). 设 $\mathcal{D}$ 为走廊阻断。若采用定义 2 对 B 类的约定, 则 $T_{\text{direct}}(\mathcal{D}) = \emptyset$, 从而对任意有限 $\theta \geq 0$ 有 $T_{\text{impact}}(\mathcal{D}, \theta) = \emptyset$ 。若此外 $\text{Seeds}(\mathcal{D}) \neq \emptyset$, 则 S^0 在 $\mathcal{D}$ 下不可行。

证明. 由定义 2, 走廊阻断不命中任何故障智能体或失效工序, 故 $T_{\text{direct}} = \emptyset$ 。 θ -跳扩张的起点为空, 故 $T_{\text{impact}} = \emptyset$ 。若存在 $r \in \text{Seeds}(\mathcal{D})$, 则 r 的占用区间与禁行窗相交。将 $\mathcal{D}$ 解释为该走廊上的强制占用后, r 与禁行窗在同一排他资源上时间重叠, 违背半开时窗无冲突条件, 故 S^0 不可行。□

命题 1 即问题层的一击反例。前半句是任务图直接集对 B 类的约定推论 (指出任务图表示覆盖不到走廊事件); 非平凡的是后半句——种子非空时排程已不可行, 因而「空影响域 $\Rightarrow$ 无需重调度」在无冲突设定下不成立。该反例不依赖任何修复算法; 实验第 6 节给出算例规模。

3.4 恢复问题

问题 1 (预约表感知的有界恢复). 给定可行排程 S^0 、扰动 $\mathcal{D}$ 与时刻 t_{now} , 求 S' , 使得: (i) S' 在 $\mathcal{D}$ 下通过无冲突校验; (ii) 未释放预约的字段尽可能与 S^0 一致; (iii) 墙钟响应时间受预算约束。次级目标为完工时间 $C_{\max}(S')$ 。

框架层将求解拆为两步: 测量——计算释放集 $U \subseteq R$; 协调——仅对 U 内运输重规划, 外侧冻结。本文测量用 $\text{Cl}(\text{Seeds})$; 对照测量用任务图影响域诱导的预约子集。

4 时空预约影响闭包

4.1 预约依赖边

定义 4 (预约依赖图). 顶点为 R 中预约。有向边 $r \rightarrow r'$ 表示「 r 失效可能迫使 r' 重规划」, 包括:

- (1) 让行边: 同走廊、异车、时间重叠或半开区间接壤 ($r.t^e = r'.t^s$), 且先进入者指向后进入者; 接壤在排他语义下合法, 但前者延后则后者必须让路;
- (2) 同车后继: 同一车辆按时间序相邻的两条预约;
- (3) 同工件后继: 同一工件工序 i 的预约指向工序 $i+1$ 的预约;
- (4) 同机后继: 同一机器时间轴上相邻两道工序所诱导的预约之间的边。

由此得到 $G_R = (R, E_R)$ 。四类边的示意图 2(b)。同工件后继连的是相邻工序所诱导的预约, 同机后继连的是同一机器时间轴上相邻两道工序所诱导的预约; 二者都以段为端点, 与第 3.1.1 节的粒度一致。

定义 5 (STRC). 对种子集 $S \subseteq R$ 、视界 H 与时刻 t_{now} , 记活预约为

$$R^\circ = \{r \in R : r.t^e > t_{\text{now}}, r.t^s < H\},$$

并令 G_R° 为 G_R 在 R° 上的诱导子图。时空预约影响闭包定义为

$$\text{Cl}(S) = \{v \in R^\circ : \exists s \in S \cap R^\circ \text{ 使 } s \rightarrow^* v \text{ 于 } G_R^\circ\},$$

即从活种子出发沿出边可达的全部活顶点(含种子自身)。若以邻接表表示 G_R° , 则可用队列在 $O(|R^\circ| + |E_R^\circ|)$ 时间内算出。

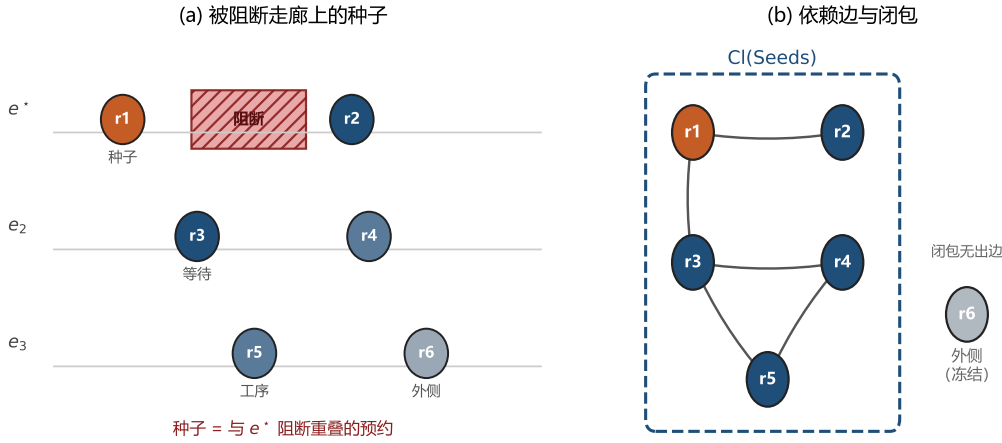


Fig. 3. 时空预约闭包构造示意。(a) 走廊阻断命中种子预约; (b) 沿让行/接续边取传递闭包 $Cl(Seeds)$, 外侧预约定向冻结。

图 3 给出玩具示意:(a) 阻断窗与种子;(b) 依赖边与闭包外壳, 闭包外顶点不是闭包内顶点的出边后继(命题 2)。

对照的任务图边界臂(记 R_1)将 T_{impact} 映射为预约释放集: 凡任务标签属于影响域且 $t^e > t_{\text{now}}$ 的预约。由命题 1, B 类上该集合为空; 闭包臂(记 R_2)取 $U = Cl(Seeds)$ 。

4.2 包含性

命题 2 (结构闭合性). 对任意 $S \subseteq R$, 在 G_R° 中不存在边 $u \rightarrow v$ 满足 $u \in Cl(S)$ 且 $v \in R^\circ \setminus Cl(S)$ 。因而, 从 $Cl(S)$ 出发沿出边无法到达闭包外的活预约。

证明. 若存在这样的边 $u \rightarrow v$, 则由 $u \in Cl(S)$ 知存在活种子 s 使 $s \rightarrow^* u$ 。拼接 $u \rightarrow v$ 得 $s \rightarrow^* v$, 故按定义 5 应有 $v \in Cl(S)$, 矛盾。□

命题 2 说明「局部」对应一个图论闭合对象: 闭包外顶点不是任何闭包内顶点的直接后继。它不声称 G_R 已枚举物理世界中一切可能耦合——边集是建模选择; 一旦边集固定, 闭合性就是定义的推论, 而不是启发式半径。

注 2 (本文不作最小性主张). 闭合性说的是闭包足够大, 没有说它不过大。二者是两个方向, 后者本文不主张, 理由有三, 按分量排列。

其一, 最小释放集的定义依赖于修复引擎, 而闭包刻意不依赖。称释放集 U 为最小, 须指明「在何种修复能力下 U 足以恢复可行」。允许改派与改序时的最小 U , 与只允许改路时的最小 U 不是同一个集合。闭包由 G_R 唯一确定, 与引擎无关——这使它在定义上不绑死某一修复启发式, 也正是它无法同时具备最小性的原因。

其二, 四类边在充分方向仍是过近似, 必要方向上已知的那条接壤通道已经补上。让行边按「同走廊、异车、重叠或接壤」建立, 只要重叠即连边, 而不判断后者是否真的只能靠等待前者通过——若该处另有旁路, 后者改道即可, 本不必进入释放集; 同车后继边同样可能把延误传不到的链尾一并纳入。这两处仍使闭包偏大。接壤一度是反方向的漏边: 半开区间首尾相接合法, 旧规则不画边, 但前者延后则后者必须让路。现已并入让行边, 差分试探在 50 对上泄漏为 $O(0/50)$; 第 7.2 节)。这只关闭了这一条已观测到的通道, 不是边集完备的证明。

其三,实测证实闭包确实不小。第 6.5 节报告,走廊阻断下闭包平均覆盖 t_{now} 之后活预约的约 90%;换言之,相对「释放全部未来预约」这一平凡上界,闭包省下的不足一成。有界修复真正省下的是不动过去与不重开搜索,而不是把未来切得很小。这一点在第 6.8 节与局限一节中再次点明,不应被「局部」「有界」这两个词误导。

不过「省下的不足一成」是释放集的规模之比,不等于闭包的价值之比:第 6.9 节把那个平凡上界作为一条独立对照臂(RA)实测后发现,正是这不足一成的差别,把改动比例从 0.604 降到 0.540(逐格 31/18/1, $p = 8.8 \times 10^{-7}$),而耗时与完工时间几乎不变。所以本注否定的是「闭包很小」,不是「闭包无用」——这两句话此前容易被读混。

因此本文的定位是:闭包给出一个由结构唯一确定、可复现、足以恢复可行的释放集,并附带闭合性保证;逼近最小释放集是另一个问题,列为后续工作(第 7.3 节)。

命题 3 (外侧字段不变的充分条件). 设释放集 $U = \text{Cl}(S)$, 并设第 1 级协调按第 5.1 节执行: (i) 禁行窗已写入路由层; (ii) 对每个 $r \in R^0 \setminus U$, 将其原字段 $(e, [t^s, t^e], k)$ 预提交到新预约表且预提交成功; (iii) 仅对 U 内运输调用路由器生成新占用; (iv) 最终排程通过无冲突校验。则在恢复后的预约表 R' 中, 每个外侧任务标签对应的占用字段与 S^0 一致。

证明. 由 (ii), 外侧预约在重放开始前已按其原区间写入新表, 且未与禁行窗冲突 (否则预提交失败)。步骤 (iii) 只为 U 内任务提交新占用, 不删除或改写已提交的外侧区间。因此, 对外侧任务标签, 新表中的 $(e, [t^s, t^e], k)$ 与预提交值相同, 亦即与 S^0 相同。校验通过仅排除新占用与既有占用冲突, 不改变已提交字段。□

注 3 (前提 (ii) 与 (iii) 各自会怎样失效). 若预提交因「外侧与禁行窗重叠」失败, 则 S 未能覆盖全部直接命中预约, 闭包种子不完备。若协调因运输-工序衔接失败而触发第 5.2 节扩域, 则新的释放集 $U' \supsetneq U$, 命题 3 只对最终未释放的外侧成立。扩域以可行性优先于最小扰动, 实验中单独报告。还有一种失效不在命题的可控范围内, 但在实现上极易发生: 前提 (iii) 要求路由器只为 U 内的运输生成新占用。若实现以比预约更粗的单位 (例如整道运输、整个工序) 决定是否重规划, 则 U 外的占用会被顺带改写, 命题的结论随之落空——不是命题错了, 而是前提没被满足。注 4 记录了本文在这一点上踩过的坑。

4.3 结构可预测性

闭包规模不是可调参数, 而是由排程与网络结构决定的量: 它随算例规模稳定上升 (第 6.2 节)。这使「局部」成为一个可以测量的量, 而不是一个人为设定的半径。

至于哪种结构会产生更大的闭包, 本文的答案是否定的, 而且否定得比「没测出来」更明确: 闭包规模不由装卸点处的静态割决定。在自建的同规模布局上, 不同边界阻断协议下的相对闭包或落在很窄的带内、或虽有方向但逐种子取值高度重叠; 换用一批不由本文设计的路网后, 装卸点最小割与漏斗占比取值恒定, 而闭包占比在路网不变、仅改机器落位时即大幅移动——一个恒定的量不可能解释一个变动的量 (第 6.7 节)。须同时说明这条否定的适用范围: 它否的是「静态割能解释闭包规模」这一具体命题, 不是「结构与闭包规模无关」。可得的公开路网数量有限, 后者尚不可判。本文因此不把闭包规模的结构可预测性列为贡献; 下一个候选刻画应是随排程而变的量, 而非纯拓扑量 (第 7.3 节)。

5 闭包上的有界修复

图 4 总览本文流程: 先以 STRC 测量释放集, 再在闭包上做第 1 级有界协调; 失败则扩域重试。对照臂 R1 仅替换释放集来源, R0+ 则打开热启动搜索——同一无冲突执行器上归因。

5.1 释放、冻结与第 1 级改路

给定释放集 U , 第 1 级协调保持原车辆指派与机器指派, 按原扫描序重放。算法 1 写出这一原语; 算法 2 在它外面套闭包与扩域。

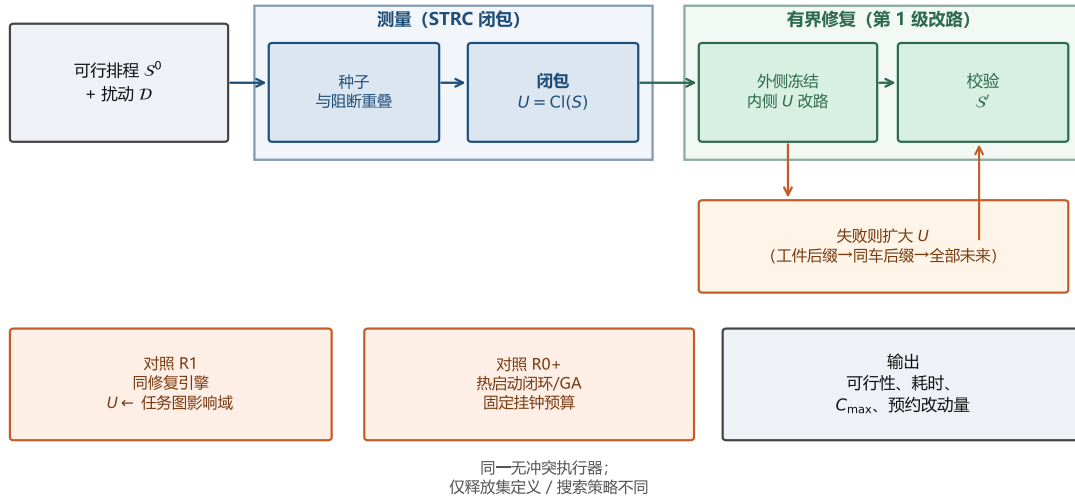


Fig. 4. STRC 有界恢复框架。上排:扰动下由种子取闭包并改路;中排:失败扩域;下排:R1(任务图边界)与 R0+(热启动重解)对照及输出指标。

算法 1 第 1 级改路重放 REPLAYREROUTE

Require: 排程 $S^0 = (x, \pi, y, R)$, 扰动 $\mathcal{D}$, 释放集 U , 时刻 t_{now}

Ensure: 排程 S' 或失败

```

1: 将  $\mathcal{D}$  写入路由层                                ▶ 阻断:禁行窗;降速;重叠段  $\tau$ 
2: 清空  $t^e > t_{\text{now}}$  的占用,得到空表  $R'$ 
3: for 每条外侧预约  $r \notin U$  且  $r.t^e > t_{\text{now}}$  do
4:   将原字段  $(e, [t^s, t^e], k)$  预提交到  $R'$ 
5:   if 与阻断窗冲突 then
6:     return 失败                                ▶ 种子未覆盖直接命中,闭包漏网
7:   end if
8: end for
9: for 每条  $r \in U$ ,按原扫描序 do
10:  已走完的走廊腿预提交;从该车在  $t_{\text{now}}$  的节点接起
11:  在当前  $R'$  上调用时间窗路由,提交新占用        ▶ 假设 A2:不改写历史
12: end for
13: 按  $\pi$  推进各机器空闲时刻与各工件就绪时刻
14: 独立校验:无冲突、无禁行窗残留、降速重叠段时长已吸收、外侧字段未改
15: if 校验通过 then
16:   return  $S' = (x, \pi, y, R')$ 
17: else
18:   return 失败
19: end if

```

闭包计算是 G_R^0 上一次广度优先,时间 $O(|R^0| + |E_R^0|)$ 。一次 REPLAYREROUTE 对 $|U|$ 个运输段各调用一次时间窗最短路;实测耗时由后者主导(第 6.4、6.9 节)。该级不打开种群搜索。同引擎消融(E3)固定算法 1,仅替换 U 来自 R1 或 R2,用以把贡献归因于边界定义。

算法 2 基于 STRC 的有界恢复(第 1 级 + 扩域)

Require: 排程 S^0 , 扰动 $\mathcal{D}$, 时刻 t_{now}

Ensure: 排程 S' 或失败

1: $S \leftarrow \text{Seeds}(\mathcal{D}); U \leftarrow \text{Cl}(S)$

2: **for** 轮次 = 0, 1, 2, 3 **do**

3: $S' \leftarrow \text{REPLAYREROUTE}(S^0, \mathcal{D}, U)$

4: **if** S' 可行 **then**

5: **return** S'

6: **end if**

7: 按扩域规则更新 U

▷ 工件后缀 / 同车 / 全部未来

8: **end for**

9: **return** 失败

5.2 失败扩域再修

先声明:扩域机制本身不是本文首创。Local Rescheduling [9] 已给出「增量式扩张时间窗直至可行」的做法,AOR 的间接影响传播亦是一种扩张(第 2.1 节)。本文的差别只在扩张沿什么走——沿依赖边而非沿时间窗——这不足以单独作为贡献。本节的作用是把闭包路径补成一个完整可用的流程,并在第 6.1 节交代它为何必须在边界消融中关闭。

当外侧冻结过紧时,算法 1 可能因运输-工序衔接等校验失败。此时不立刻求助于全局搜索,而按下列顺序扩大 U 并重试同一引擎: (i) $U \leftarrow U \cup \{\text{释放集内各工序的同工件后缀未来预约}\}$; (ii) $U \leftarrow U \cup \{\text{所涉车辆的同车后缀未来预约}\}$; (iii) $U \leftarrow \{r \in R : r.t^e > t_{\text{now}}\}$ (释放全部未来)。扩域仍是确定性单遍协调。实验表明扩域可将可行率拉至全样本,耗时仍保持在毫秒至十余毫秒量级(第 6 节)。需要说明的是,耗时随扰动比例 φ 并不单调(表 11):比例小时初始闭包偏紧、冻结偏多,扩域更常被触发;比例大时初始释放已较松、往往一轮成功,但单轮要重规划的运输更多。两种效应方向相反,故本文不宣称耗时随 φ 有可预测的走向,只宣称它不随 φ 崩溃。

5.3 与全局重解的对照

对照臂在相同禁行约束下做固定挂钟预算的热启动搜索(记 R0+)。内部把 (x, π) 编成一条染色体;一次解码指:按该染色体的指派与扫描序,在装了禁行窗的预约表上从头为全部运输调用第 3.1.2 节的时间窗路由,得到新的 R 与 C_{max} 。R0+ 重复解码并允许改染色体;第 6.9 节的 RD 臂只解码一次且不改染色体。R0+ 的解码受假设 A2 约束:已完成占用预提交,只从 t_{now} 接着排未开始的工序(第 6.8 节)。RD 与 R0+ 走同一套固定前缀解码,只解码一次且不改染色体。对照取热启动而非冷启动,是因为对照必须取强者:冷启动会白白丢掉原解携带的全部信息,赢它不说明问题。R0+ 可改派与改序,因而更可能改善 C_{max} ,但耗时由预算决定。

角色分工是:STRC 路径优先可行性与响应时间;R0+ 优先完工时间。需要预先说明这个「分工」的确切含义——它不是指存在某个预算区间使闭包路径的 C_{max} 更优。第 6.8 节的实测表明这样的区间不存在:R0+ 在最小预算下已占优,两条曲线不相交。分工指的是两者被不同的约束选中——当响应必须在毫秒量级时,只有闭包路径可用;当允许秒级停机时,应当用 R0+。「外侧字段逐条不变」只在命题 3 的条件下保证,实验中作部分观测。

只与 R0+ 对照会留下一个反问:毫秒量级那一档里,是否根本不需要边界?把「不划边界」的做法直接实现出来即可回答——全局右移不判定谁受影响,只把未完成的一切统一后推;而「重算但不问边界」这一端,最便宜的形态是拿原染色体重解码一遍。两者都在毫秒量级,构成 STRC 在同一响应档位内的真正对手,读数在第 6.9 节。

算法 2 与图 4 对应:闭包界定 U , 第 1 级改路,失败则按工件/车辆/全部未来依次扩域。

6 实验

6.1 协议

执行器与算例. 下层路网、时间窗路由与独立校验器固定。扩样矩阵使用**五个算例**: 小例 example_3x3x2、拥堵例 congested_8x4x4, 以及三个 S8x4x4 布局 high、funnel 与 mid。后三者同规模、同柔性, 只差走廊拓扑, 因此它们之间闭包规模的差异可归因于拓扑而非问题尺寸。种子取十个, 即 42、7、2024、99、123、13、1、777、31415 与 8, 共 50 个算例-种子对。初始排程由第 3.1.2 节的时间窗路由按扫描序重放得到, 并通过独立无冲突校验; 同一算例-种子对下各对照臂共享该初始解。本文不依赖这份 S^0 的搜索来源, 只要求它在扰动前可行。

第二批: 布局出外外部化. 上面五个算例的布局都是本文设计的, 其中三张同规模布局还同属哑铃一族(彼此只差装卸点出口与中段的并行通道数, 即只差容量、不差几何)。为了让布局不再由本文决定, 另跑一批 5 个算例: 路网的网格尺寸、装卸站与机器落位、断掉的边这三项逐项转录自 Lyu 等 [12] 附录 A 的布局图, 转录结果由脚本与 van Os 配套工具集所给的机读编码逐字段对账。机器台数因此由布局决定而非由本文选定, 为 4 至 8 台。种子仍取同样十个, 合计 50 个算例-种子对。两批算例逐参数同口径: 工件数、AGV 数、每工件工序数、 H 、 F 以及运输/加工时长比的标定目标一律相同, 于是两批之差可以干净地归因给布局出外。

该附录共给出六张布局(3 至 8 机)。3 机那张未用: 柔性度 F 的含义是平均候选机数占机器总数之比, 本文固定 $F = 0.6$, 在 $|M| = 3$ 时给出平均候选机数 $F \cdot |M| = 1.8 < 2$, 无法在满足「每道工序至少两台候选机」的前提下达到该柔性目标; 为一张布局放宽 F 等于多引入一个变量, 故宁可少用一张。

其余五张只落在 2 张不同路网上, 须先说明: 4 机与 5 机两张共用同一张 4×4 网格(去掉同一条边), 6、7、8 机三张共用同一张 5×5 网格(去掉同两条边); 五者彼此的差别在于机器台数与落位, 而非路网几何。这是原始发布件本身的组织方式, 不是本文的取舍。因此第 6.7 节读这批数时, 路网几何这一维的独立样本是 2 而不是 5, 而机器落位这一维有 5 个; 下文的论断严格按这个划分来下。

有三处口径必须先声明清楚。**其一, 逐段行驶时间不是原始数据.** Lyu 只为单个示例算例发表过逐段时长, 且其取值并不均匀; 附录 A 那批布局的逐段时长从未发表。故这里按「所有边等权」补齐——它与 van Os 模型里「单步时长为常量」的假设在结构上一致, 只是时间单位的标度不同。因此本批结果不可与 Lyu 或 van Os 的参照值作任何比较; 它提供的只是一批不由本文设计的拓扑, 而不是一个对标集。**其二, 装卸点做了合并.** 原布局有分离的装货站与卸货站, 本文只有一个装卸点, 故取车辆起始处的装货站充当该点, 卸货站退化为普通网格节点。其三, 工件数据没有一并借用。已实测的公开族里平均运输时长只占平均加工时长的百分之几, 运输在其上几乎不构成争用; 若把工件数据也搬进来, 走廊争用连同以它为对象的整套结论都会退化到观察不到。外部化的只有路网。同一工具集另附的一族布局(其目录标注为 Liu 2023)未收入: 这些布局的数据声明允许对角移动, 而本文的走廊是四邻接的; 接受对角移动要改的是下层路由层而不是算例。

扰动. 除非另注, $t_{\text{now}} = 0.35 C_{\text{max}}(S^0)$ 。E1-E3/E5 在繁忙走廊上施加单窗阻断; E6 在同一 t_{now} 上另加三类扰动; 规模扫描将未来工序按最早未来预约排序, 取前比例 ϕ 的预约时窗作为微阻断(不合并)。

对照臂. R1: 任务图影响域 + 第 1 级改路, 全文取 $\theta = 2$ (该值只影响 A 类扰动下 $|U_{R1}|$ 的大小; 由命题 1, B 类上任何有限 θ 都给空集); R2/STRC: 闭包 + 第 1 级改路(规模扫描打开失败扩域; E3/E5 关闭扩域); R0+: 热启动闭环搜索, 固定挂钟预算; RS: 全局右移, 不划边界且不重路由(第 6.9 节); RD: 原染色体重解码, 即本实现里最便宜的一次全局重算(同节); RA: 不划边界但保留重路由——释放 t_{now} 之后的全部预约, 其余与 R2 逐项相同, 用于把闭包这一项单独隔离出来(同节)。这五条臂按「是否划边界 $\times$ 重算多少」铺开: R1 与 R2 只差边界定义, RS 不划边界而整体后推, RD 与 R0+ 则重算全局而不问边界, 只差搜索多少代。

一处必须声明的口径. E3 中必须关闭失败扩域。若开着扩域, R1 会靠「一路扩到释放全部未来预约」而偶然成功, 测到的就不再是边界定义的差别, 而是扩域规则的兜底能力。扩域本身有效(规模扫描中它把可行率拉

表 3. E1 漏报(走廊阻断;每算例 10 种子,共 50 对)。|R| 为全表预约数,CI/|R| 为闭包占全表的比例。均值取多种子平均。后三行同规模同柔性,只差走廊拓扑。

算例	种子数	C1 通过	均 R	均 Seeds	均 CI	CI/ R
example_3x3x2	10	10/10	28.6	5.6	16.9	0.59
congested_8x4x4	10	10/10	85.0	15.0	48.4	0.57
S8x4x4 (high)	10	10/10	150.5	15.5	88.3	0.59
S8x4x4 (funnel)	10	10/10	149.1	15.5	85.4	0.57
S8x4x4 (mid)	10	10/10	147.6	15.0	82.8	0.56
合计	50	50/50	—	—	—	—

满),但它与被测变量正交,故在边界消融中一律关闭。该端点并未因此被回避:它就是第 6.9 节的 RA 臂,那里作为独立对照臂逐格测了它的代价。

两批算例各自承担什么. 两批不是同一组证据的两次采样,它们回答的问题不同,故本章各节只由其中一批支撑。主批次回答现象是否存在、机制是否干净:B 类扰动下任务图边界为空而原排程不可行(第 6.2 节)、闭包对该边界的包含与闭包外字段的不变性(第 6.3 节)、同一引擎下仅替换边界定义的消融(第 6.4 节)。这三件事都要求对算例参数有完全的控制权,尤其是运输/加工时长比——它决定走廊争用是否成立——而公开族给不了这种控制权(第 7.2 节局限第 4 条)。第二批回答的是另一个问题:这些现象是否只在本文自选的布局上成立。它因此只用于复现,不产生新的性能读数(第 6.12 节)。

两处需要点明。其一,第二批在本文中只用于收窄主张,未曾用于支持任何一条:它唯一改写的结论是结构可预测性,且改写的方向是让本文少主张一件事(第 6.7 节)。其二,只有 E4 一格必须两批合读:主批次的三张同规模布局同属一族、只差容量不差几何,在它们身上“指标太粗”与“算例集几何差异不足”这两种解释混在一起;而外部布局若没有逐参数同口径的自建对照,也无从判断其闭包占比的变动幅度算不算大。其余各节的结论由主批次单独成立,第二批只增强其外部效度。

6.2 E1:任务图漏报

表 3 与图 5(左)汇总命题 1 的扩样结果: 50/50 样本上 $T_{\text{impact}} = 0$ 、种子与闭包非空、阻断后原排程不可行,结构泄漏均为 0。

按命题 1 之后那段话的提醒,读这张表时重量应压在后半句上。 $T_{\text{impact}} = 0$ 这一列是定义 2 对 B 类约定的推论,它把「任务图这一表示覆盖不到走廊事件」显示出来,但本身不是发现。非平凡的是 feasible_after_block 一列恒为假——排程确实已经不可行。两者合起来才构成反例。

闭包规模随算例规模上升(16.9 $\rightarrow$ 48.4 $\rightarrow$ 80 余),但 CI/|R| 在五个算例上稳定在 0.56–0.59,未能区分三个 S8x4x4 布局。这与第 6.7 节的结论一致:本文没有测出一个能可靠预测闭包规模的结构指标,故不把「闭包规模由布局决定」列为贡献。

6.3 E2:包含性

结构包含性(E2a)在 50/50 样本上成立(泄漏 = 0),与命题 2 一致。第 1 级修复(无扩域)在 50/50 样本上可行。外侧字段逐条不变(E2b / 命题 3 的观测形式)同样在 50/50 样本上成立(表 4)。

需要区分 E2a 与 E2b 说的不是一回事,满通过并不使命题 3 升格为充要。E2a 是图上的性质,由命题 2 保证,满通过是应当的;若不满通过,说明实现与定义不符。E2b 是修复之后的性质,它检验的是命题的四条前提在本文协议下是否成立——而不是这些前提可以省略。特别地,前提 (ii) 要求外侧预约逐条预提交成功;若某条外侧预约

表 4. E2 包含性(10 种子/算例)。E2a 为结构闭合(泄漏 = 0), E2b 为闭包外侧预约字段逐条不变。

算例	E2a 闭合	第 1 级可行	E2b 外侧逐条不变
example_3x3x2	10/10	10/10	10/10
congested_8x4x4	10/10	10/10	10/10
S8x4x4 (high)	10/10	10/10	10/10
S8x4x4 (funnel)	10/10	10/10	10/10
S8x4x4 (mid)	10/10	10/10	10/10
合计	50/50	50/50	50/50

与禁行窗重叠,预提交即失败,那说明种子集不完备(注 3),此时命题不适用。本文的实验落在前提成立的一侧,不等于任何设定下都会落在那一侧。

注 4(一处曾经把实现缺陷读成边集缺陷的教训)。这一栏在早前的批次上只有 24/50。当时的解读是「依赖边集未能覆盖若干弱耦合」,并据此把加细边集列为首要后续工作。复核后该解读是错的。释放集是一个预约集合,而重放时是否改路却按工序判定——同一工序的空载段与满载段只要有一段落在闭包内,整道运输就被重规划。于是当只有满载段进入闭包时,同工序的空载段也被改写,而它的任务标签并不在释放集内,遂被记为外侧漂移。26 个不通过格上的 84 条漂移预约无一例外都是这种同工序空载段,其中 71 条(85%)在 t_{now} 之前就已执行完毕——改写它同时违反假设 A2。改为按段判定(已执行完或未被释放的空载段沿用原计划,只重规划真正落在释放集内的段)之后,漂移归零。换言之,此前测到的不是边集对物理耦合的覆盖度,而是命题 3 前提 (iii)「仅对 U 内运输调用路由器」在实现上没有被真正满足。记下这一条,是因为它示范了一种容易发生的误判:把实现与定义的粒度错配,读成模型对世界的刻画不足。

同一类错配还剩一层,而 E2b 看不见它。按段判定之后,一个跨越 t_{now} 的运输段仍是整段重规划:只要它有一条走廊腿的结束时刻落在 t_{now} 之后,整个段的任务标签就进入释放集,于是它在 t_{now} 之前已经走完的那几条腿也被重排——等于把车退回该段起点重走一遍,违反假设 A2。E2b 查不出这一情形,因为这些腿的任务标签在闭包内,本就不在外侧字段的比对范围里。用一项独立的「历史不可改写」审计(逐条比对 $t_{\text{end}} \leq t_{\text{now}}$ 的预约)才测出来:50 个格子里有 36 个存在此类改写,合计 86 条。把粒度再细一层——按走廊腿切分,已走完的腿原样保留,改路只从车在 t_{now} 时实际所处的节点往后接——之后该审计在全部 50 个格子上归零,且各项通过率不变、 C_{max} 在 6 个格子上变好、无一格变差。本文报告的是修正后的读数。

6.4 E3:边界消融

表 5 与图 5(右)固定第 1 级改路并关闭失败扩域。B 类上 50/50 样本 R1 释放集为空且不可行, R2 全部可行。

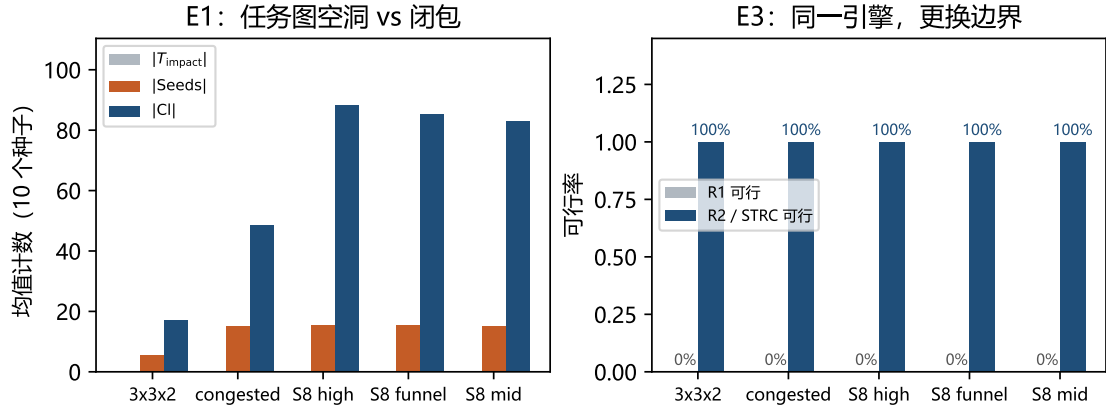
这是本文最强的一组对照,但它的说服力来自干净而不是来自数字大:同一个修复引擎、同一份初始解、同一个 t_{now} 、同一批种子、同一段阻断窗,唯一的差别是释放集合怎么算。因此 0/50 对 50/50 这个反差不能归因于改路启发式的优劣——R1 侧根本没有触发任何改路,它的释放集是空的,第 1 级改路对空集是恒等操作,于是原编程原封不动地留在被阻断的走廊上。换句话说, R1 的失败方式不是「修得不好」,而是「判断为无需修」。修复耗时中位 3.2 ms、最大 12.8 ms。

6.5 E6:扰动类型与边界定义交叉

E1-E3 只用了走廊阻断一种扰动。本小节把扰动类型这一维展开,在同一 50 对上另加路段降速、车辆故障与机械臂故障三类,给出表 6。

表 5. E3 边界消融(无扩域;5 算例 $\times$ 10 种子)。miss_B 指该格上 R1 释放集为空而 R2 非空。

算例	miss_B	R1 可行	R2 可行	均 $ U_{R2} $	R2 耗时/ms
example_3x3x2	10/10	0/10	10/10	16.9	0.66
congested_8x4x4	10/10	0/10	10/10	48.4	2.84
S8x4x4 (high)	10/10	0/10	10/10	88.3	3.40
S8x4x4 (funnel)	10/10	0/10	10/10	85.4	5.09
S8x4x4 (mid)	10/10	0/10	10/10	82.8	3.96
合计	50/50	0/50	50/50	—	—

Fig. 5. E1/E3 扩样可视化(5 算例 $\times$ 10 种子均值)。左:任务图影响域为空而种子与闭包非空;右:同改路引擎下仅换边界时 R1 可行率为 0、R2 为 100%。表 6. E6 扰动类型 $\times$ 边界定义(50 对/类,共 200 次边界测量)。 $|R^c|$ 为 t_{now} 之后的活预约数。「R1 为空」即任务图边界判该扰动无需重调度的格数。末列不开扩域:B 类为第 1 级改路,A 类为换车或改派。

扰动	类	R1 为空	均 $ U_{R1} $	均 $ CI $	$CI/ R^c $	R2 可行
走廊阻断	B	50/50	0.0	64.4	0.92	50/50
路段降速	B	50/50	0.0	64.4	0.92	50/50
车辆故障	A	0/50	31.1	60.6	0.85	50/50
机械臂故障	A	0/50	42.8	65.9	0.94	50/50

先说测什么。边界维四类都报:任务图影响域有多大、预约闭包有多大、后者是否包含前者。修复维四类都开口。B 类仍只改路:阻断把禁行窗写入预约表,降速在窗内按倍率 $\tau_{\text{mult}} = 2$ 降低进度、窗外仍走原 τ ,走廊仍可走。A 类额外换车或改派,不改扫描序;扩域开关对它们不改变动作集合。

四点读数。第一,A/B 二分在 200 次边界测量上无一例外:两类 B 扰动的任务图边界全部为空,两类 A 扰动全部非空。第二,在全部 100 次 A 类测量上 $U_{R1} \subseteq CI$ ——这说明改用闭包边界在 A 类上不会丢失任务图能看见的东西。包含在 87/100 上是严格的,其余 13 格两者恰好相等(此时闭包沿依赖边没有走出任务图影响域诱导的那个集合),不是包含失败。代价是释放集平均变大:车辆故障 $31.1 \rightarrow 60.6$,机械臂故障 $42.8 \rightarrow 65.9$ 。结合第一点,闭包边界在这四类扰动上是任务图边界的一致加强,而非另一种取舍。第三,降速与阻断的边界两行逐格相同,在全部 50 对上闭包规模一致。这不是两次独立的边界验证:种子规则对二者都是「与失效时窗重叠的活预约」,故它们

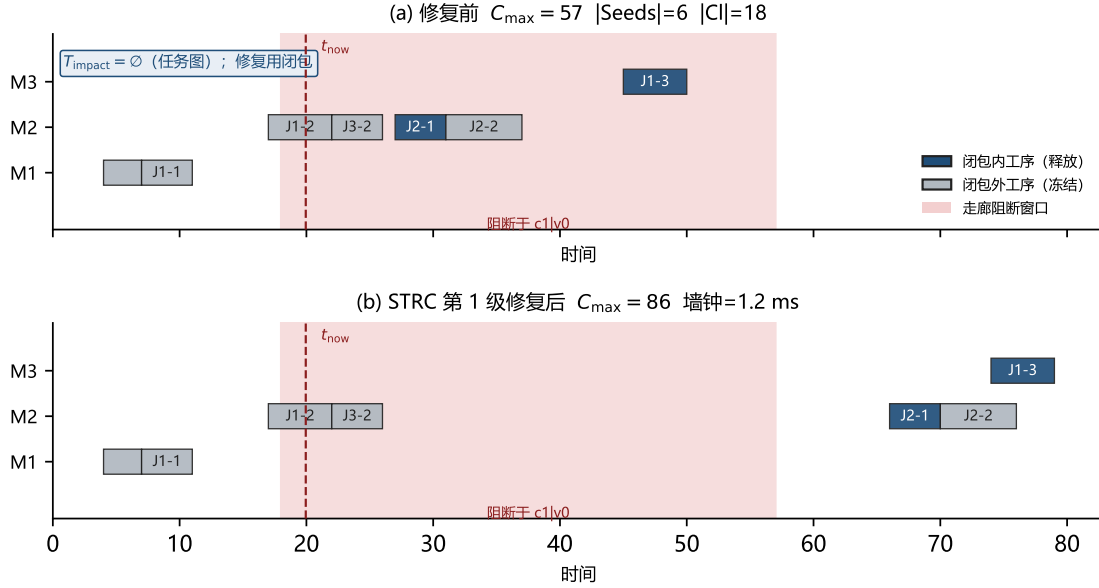


Fig. 6. 案例甘特(example_3x3x2, seed 42)。 (a) 阻断前原排程,按是否属于 CI 着色; (b) STRC 第 1 级修复后。任务图影响域为空;蓝色工序对应闭包内运输被改路。

按构造给出同一个种子集,进而给出同一个闭包。该行证实的是任务图边界对降速同样为空。第四,修复维四类均满。阻断 50/50;降速同口径 50/50。降速一度把与窗有交的穿越整段乘 2,13 格死在「占用偏短」——窗内 2τ 无空档时,进入时刻被拉回窗内却仍按原 τ 落盘;其中 12 格连释放全部未来也过不了,故不是冻结外侧把降速车卡死。改为只缩放重叠进度之后,第 1 级满通过,扩域不再被触发。走廊仍可走,与「整段封死再绕路」的物理差别还在,只是不再被整段倍率放大。车辆故障换车 50/50,机械臂故障改派 50/50。A 类不走释放集扩域,开关不改变动作集合。本文不把换车与改派当作创新,只把覆盖矩阵的修复维补上。

这张表也给出了不作最小性主张的实测依据(注 2)。走廊阻断下闭包覆盖 t_{now} 之后活预约的约 92%;相对「释放全部未来」这一平凡上界只省下一成。有界修复真正省下的是不动过去与不重开搜索,而不是把未来切得很小。第 6.2 节按全表统计的 $CI/|R| \approx 0.57$ 与此不矛盾:全表包含 t_{now} 之前已经执行完毕、按假设 A2 本就不该改的那一半。两个口径都给出,以免读者只看到有利的那个。

6.6 案例甘特

E1-E3 给出的是跨种子的计数与可行率。为把「任务图覆盖不到、闭包可修」落到时间轴上,图 6 取 example_3x3x2、种子 42 的一例走廊阻断,协议同 E2/E3:决策时刻取 $0.35 C_{\max}$,繁忙走廊单窗阻断,第 1 级改路、无扩域。

该例上 $T_{\text{impact}} = \emptyset$,而 $|Seeds| = 6$ 、 $|CI| = 18$ 。图 6(a) 为扰动前原排程的机器甘特:蓝色工序的运输预约落入闭包(将被释放改路),灰色工序在闭包外(预提交冻结);红色虚线为 t_{now} ,浅红带为阻断时窗。图 6(b) 为修复后:闭包内工序时间轴重排,排程在阻断下恢复可行, $C_{\max} : 57 \rightarrow 86$,耗时亚毫秒级。本图不用于宣称完工时间最优——它只说明统计结论对应何种局部改动形态;完工时间与全局重解的权衡见随后的 E5 与规模扫描。

表 7. E4 闭包占比 $Cl/|R|$ (LU 割边阻断;种子 42, 7, 2024, 99, 123)。本表算例为 12 工件 / 8 机 / 12 车、运输与加工时长比标定为 4.0。该口径与表 8 不同(后者为 8 工件 / 4 机 / 4 车、比值 1.0、十种子),两表虽共用 funnel/high 这两个布局名,数值不可跨表比较;表 8 内部才是同口径对照。

布局	42	7	2024	99	123	中位
funnel	0.254	0.181	0.296	0.309	0.180	0.254
high	0.156	0.204	0.372	0.229	0.184	0.204

6.7 E4:结构(探索)

在 LU 割边阻断下,funnel 与 high 的闭包占比随种子波动(表 7)。方向上确实是 funnel 更高——中位 0.254 对 0.204,均值 0.244 对 0.229——但两组取值高度重叠:funnel 落在 $[0.180, 0.309]$,high 落在 $[0.156, 0.372]$,后者的区间把前者整个包住,且单个种子即可反向(2024 一列上 high 的 0.372 反高于 funnel 的 0.296)。五个种子、两个布局,这个差距不足以支撑任何结论。第 6.2 节按繁忙走廊阻断协议在三个同规模布局上得到的 $Cl/|R|$ 亦落在 0.54–0.57 的窄带内,同样未能区分布局。

换一批不由本文设计的布局,这条负结果的成因才分得清。上述三张同规模布局同属哑铃一族,彼此只差并行通道数,也就是只差容量、不差几何;于是“指标太粗”与“算例集本身几何差异不足”这两种解释在它们身上是混在一起的,无从分辨。第 6.1 节第二批把两者分开了(表 8)。

结果的方向与预期相反,而这恰是关键。按第 6.1 节的声明,这批算例在路网几何上只有 2 个独立样本,故不能用它来论证“闭包占比随路网几何变化”;能论证的是另一件事,而且更有力。在同一张 4×4 路网上,只把机器台数与落位从 4 机换成 5 机,闭包占比中位即由 0.512 落到 0.305,极差 0.207;同一张 5×5 路网上换三种落位,极差 0.075。作为对比,逐参数同口径的三张自建布局——它们是三张不同的路网——只落在 0.446–0.497,极差 0.051。即:路网固定、只动机器落位,闭包占比的变动幅度可以超过换三张不同路网的幅度—— 4×4 那张上是约四倍, 5×5 那张上约一点五倍。两张路网上都成立的是不等号的方向,倍数本身只有两个样本,不宜细读。

于是静态割被干净地否证了。在这 5 个外部算例上,LU 最小割恒为 2、漏斗占比恒为 0——两张路网都把装卸站放在网格角点,而四邻接网格的角点恰有两条出边,这个割因此被钉死。同一工具集另附的那族布局(即因允许对角移动而未收入的一族)把装卸站放在网格边的中点,其出边数是 3——换言之,这个指标在真实布局上只取到极少数几个由“装卸站摆在哪种网格位置”决定的值,并不随几何连续变化。关键在于:这两个指标在上述整段变动中一动不动,而被解释的量已经变了 0.207。一个取值恒定的量不可能解释一个如此变动的量,所以它们连“相关性弱”都谈不上,是无从谈起;第三个指标(每节点走廊数)虽有两个取值,与闭包占比的秩相关为 0。

据此可把原先那句含糊的观察收紧为一条明确的否证:闭包规模不由装卸点处的静态割决定。最小割这一族指标本是围绕哑铃布局那个可调的 LU 咽喉设计出来的,迁到真实布局上就退化成常量,不具备解释力。因此本文不把「闭包规模由争用结构决定」列为贡献。更进一步,上面那组读数指出了替代方向:既然路网不变而落位一变,闭包占比就大幅移动,那么决定闭包规模的不是路网能提供多少条通路,而是需求落在哪里——应当刻画的是 t_{now} 邻域内走廊占用的时序密度,而不是静态拓扑的连通度(第 7.3 节)。

6.8 E5:与全局重解的权衡

本小节的设计意图原是找一个交叉点:预算紧时有界修复占优,预算放宽后热启动全局重解反超。该交叉点不存在,本文如实报告这一结果,并据此改写机制层的定位。

表 9 与图 7 在两个算例上对种子 {42, 7, 2024} 扫描 R_0+ 预算 {0.2, 1, 2} s,共 $2 \times 3 \times 3 = 18$ 个预算点。 R_0+ 即使在最小预算 0.2 s 下也已优于 R_2 ;加大预算几乎不再拉开(小例三档持平,拥堵例再降约 4)。在全部 18 个点上无一例外。

表 8. E4 布局出外外部化对照(LU 割边阻断,十种子;CI/|R| 取中位)。全表逐参数同口径,只差路网与机器落位。**外部算例只落在 2 张路网上**($N_1:4 \times 4$ 去一条边; $N_2:5 \times 5$ 去两条边),组内各行共享同一路网,只差机器台数与落位;“极差”一列按组内给出。要点: N_1 组内路网不变、仅换落位即得极差 0.207,已是三张不同 自建路网之间极差 0.051 的约四倍;而 LU 最小割与漏斗占比在整个外部批 取值恒定,故无从解释这些差异。边权为本文补齐的等权值,故本表不与外部文献的参照值比较。

出处/路网	算例	机器	节点/走廊	LU 割	漏斗占比	CI/ R	组内极差
外部 N_1	LyuL2	4	16/23	2	0	0.512	0.207
外部 N_1	LyuL3	5	16/23	2	0	0.305	
外部 N_2	LyuL4	6	25/38	2	0	0.385	0.075
外部 N_2	LyuL5	7	25/38	2	0	0.422	
外部 N_2	LyuL6	8	25/38	2	0	0.460	
自建	high	4	10/10	2	0.286	0.497	0.051
自建	funnel	4	9/8	1	0.286	0.491	
自建	mid	4	11/12	2	0.286	0.446	

表 9. E5 与受 A2 约束的热启动全局重解(3 种子均值)。「改动比例」为被改写的预约占全表之比。 $C_{\max}$ 一列 R0+ 全区间更优。

算例	预算/s	R0+ $C_{\max}$	R2 $C_{\max}$	改动比例 R0+ / R2
congested_8x4x4	0.2	279.3	299.0	0.61 / 0.57
	1.0	276.0	299.0	0.61 / 0.57
	2.0	275.7	299.0	0.61 / 0.57
example_3x3x2	0.2	78.3	88.7	0.60 / 0.60
	1.0	78.3	88.7	0.60 / 0.60
	2.0	78.3	88.7	0.60 / 0.60

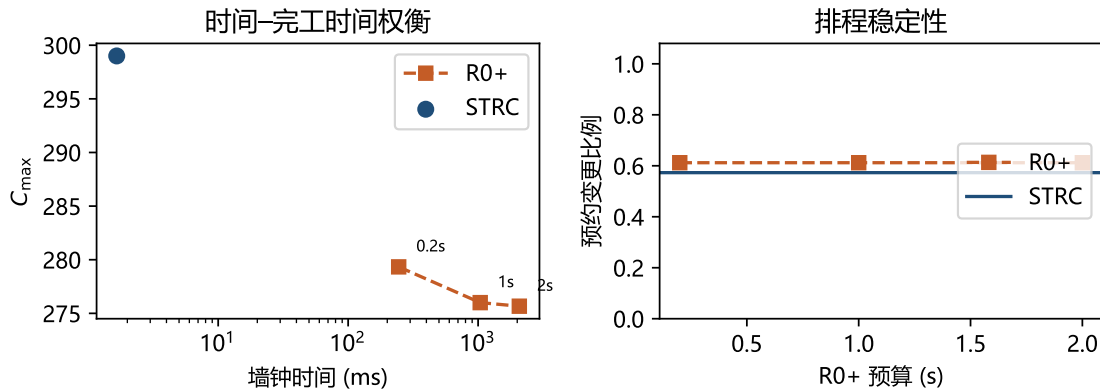


Fig. 7. E5:受 A2 约束的 R0+ 预算扫描与 STRC 定点(congested_8x4x4, 3 种子均值)。左:耗时- $C_{\max}$,两条曲线不相交;右:预约改动比例,两臂同在六成附近。

据此改写角色分工的表述。有界修复在时间上低两至三个数量级,在解质量上全区间劣势;与 R0+ 的改动比例则已同在 50%–66% 量级(稳定性须看第 6.9 节与 RA 的对照)。诚实的定位是:

表 10. E7 便宜对照臂(50 对,协议与 E3 相同:繁忙走廊单窗阻断、 $t_{\text{now}} = 0.35 C_{\text{max}}^0$ 、R2 不开失败扩域)。四臂均 50/50 可行。「A2 越界」为改写了已完成预约的格数;「释放集」为该臂释放的预约占 t_{now} 之后活预约的比例;未列为该臂对 R2 的 C_{max} 胜负。RA 与 R2 只差释放集,故这两行之差可全部归因于闭包本身。本表不可与表 9 并读——那批只有两个算例。

臂	A2 越界	ms(中位)	平均退化	改动比例	释放集	对 R2 赢/平/输
RS 全局右移	0/50	1.42	58.8%	0.627	—	7/16/27
RD 重解码	0/50	4.16	49.0%	0.618	—	22/14/14
RA 不划边界	0/50	2.76	49.9%	0.604	1.00	0/46/4
R2 闭包有界修复	0/50	2.74	49.8%	0.540	0.92	—

有界修复不是「更好的重调度」,而是「必须立刻给出可执行方案时」的那一档。若允许秒级停机,受 A2 约束的热启动全局重解在完工时间上仍占优,但优势幅度已远小于从 $t = 0$ 重排的那一档。

需要说明为什么不换指标绕过去。若改用「相对初始解的 C_{max} 退化率」或「 C_{max} 与改动比例的加权和」,R2 都会好看一些,但那是在事后挑一个对自己有利的标量化方式。 C_{max} 是本领域的主指标,本文在它上面偷了,就按它报。这条读数也约束了前文的用词:第 5.3 节所说的「分工」,指的是两者在响应时间这一约束下各自占据的区间,不是指存在某个预算区间使 R2 更优。

本表的 R0+ 已受假设 A2 约束:解码把 $t_{\text{end}} \leq t_{\text{now}}$ 的占用预提交,只从决策时刻接着排未开始的工序,并允许改染色体。历史不可改写审计上,小例与拥堵例各 9 个预算点越界均为 0;同工件后道已出车、前道还要重定时,改为强制原车、只重规划 t_{now} 之后的尾段,历史空载前缀得以保留。有界修复在同样 6 个算例-种子格上仍一律为零。与「从 $t = 0$ 重排」的旧对照相比, C_{max} 差距大幅收窄,改动比例不再是百分之百对六成;但 18/18 仍无交叉。缺口里原先混着的「改写历史」已经被拿掉,剩下的是全局改派改序相对单遍改路的那一截。

6.9 E7:便宜的可采纳对照

上一节的对照臂是热启动种群搜索,预算 0.2~2 s。只用它作对照会留下一个显然的反问:「低两至三个数量级」这个读数,有多少来自闭包本身,又有多少只是因为对照选贵了?本节把阶梯中间那两档便宜的全局重算补上。

- **RS(全局右移)**。不改路径、不改指派、不改扫描序,把未完成的一切统一后推一个延迟 Δ , Δ 取到刚好让受阻走廊上所有可动的占用落到阻断窗之后。冻结判据与 R2 相同($t_{\text{end}} \leq t_{\text{now}}$ 不可动),故它按假设 A2 是可采纳的;因为是统一平移,一切间隔要么不变、要么变大,而全部约束都是「 $\geq$ 」型,无需重新路由即得可行解。它是「不划边界」的一端:连重路由都放弃(第 2.1 节)。
- **RD(原染色体重解码)**。保持机器指派与扫描序,在装了阻断的路由层上走与 R0+ 同一套固定前缀解码。它是本文实现里最便宜的一次全局重算——种群搜索每评估一个候选都至少要付这笔钱。它不改染色体,因而比 R0+ 更便宜,也更少协调自由度。
- **RA(不划边界,但保留重路由)**。释放 t_{now} 之后的全部预约,再走与 R2 完全相同的改路重放。它是「不划边界」的另一端:与 RS 不同,它保留了重路由这一动作,因而与 R2 只差释放集这一项。它与 R2 共用修复引擎、共用冻结判据、共用阻断安装,唯一差别是释放集取了平凡上界而非闭包。因此它是本文唯一能把闭包这个对象单独隔离出来的对照:两臂之差不含引擎、协议或实现的贡献。它同样按 A2 可采纳,且已是最大释放集,无需失败扩域。

设置 RA 的必要性来自一个尖锐的反问。闭包既然已经覆盖了活预约的约 92%(注 2),那么划这条边界还剩下多少用? E3 比较的是两种不同的边界定义(任务图 vs 时空闭包),而 RA 问的是更根本的一层:要不要边界。

表 10 给出四条读数。

其一,在给出可行方案的各臂中,唯一比有界修复更快的是全局右移,而它在解质量与稳定性上都输。(任务图边界臂 R1 更快,但它在 0/50 上恢复可行,不构成候选。)RS 只要 1.42 ms,约为 R2 的一半;代价是完工时间平均退化 58.8%(R2 为 49.8%),逐格对比 27/16/7(R2 赢/平/输),且改动的预约反而更多(0.627 对 0.540)——统一平移把每一条未来预约的时刻都改了。值得指出的是,若想把右移做得有选择——只推迟真正受影响的车,其余不动——就必须先知道哪些预约受影响,那正是本文定义的那个边界。右移之所以能不划边界,恰恰因为它放弃了选择。

其二,最便宜的一次全局重算比一次有界修复更贵。RD 的耗时中位是 4.16 ms,为 R2 的 1.5 倍。这条读数与搜索配置无关:任何基于解码的搜索,每评估一个候选都要付至少一次解码的代价。因此上一节「低两至三个数量级」并不是选贵了对照造成的——把对照换成同一实现里最便宜的那一档,有界修复仍然更快。

其三,受 A2 约束后 RD 仍略优,但改动更多。全表看 R2 对它是 14/14/22(赢/平/输),平均退化 49.8% 对 49.0%。A2 越界已是 0/50:降级后道改为强制原车、只重规划决策时刻之后的尾段,历史空载前缀不再丢。故「RD 更好」不能写成「来自改写历史」——这是同染色体、只排未来时,全局重路由相对单遍改路的那一截。改动比例 RD 为 0.618,高于 R2 的 0.540:稳定性仍在闭包一侧。

其四,闭包换来的是稳定性,而且只换来稳定性。RA 与 R2 只差释放集,因此这一对读数直接回答了「划这条边界值不值」。闭包平均只比平凡上界少放 $1 - 0.92$ 的活预约(逐格 0.80–1.00,取到 1.00 的 4 个格上闭包就是全部未来预约,两臂恒等)。就是这不足一成的差别,换来了改动比例从 0.604 降到 0.540:平均降 0.064、中位降 0.029,逐格 31/18/1(R2 更稳/持平/更差),Wilcoxon 符号秩双侧 $p = 8.8 \times 10^{-7}$ 。方向高度一致,幅度不大——唯一的反例格降幅为 +0.007,可忽略但如实记录。与此同时耗时几乎相同(2.74 对 2.76 ms),完工时间也几乎相同(比值均值 0.999,逐格 R2 赢/平/输 4/46/0、从未更差,最占优的一格为 0.970)。

这三项各自都该如此,合起来才是重点。耗时相同,说明「毫秒级响应」这个读数不来自闭包,而来自「单遍改路、不重开搜索」——这一点本文就不主张新颖(表 1)。完工时间相同,说明闭包不是一种优化手段。改动比例显著更低,说明闭包兑现的恰是命题 3 承诺的那个量:外侧字段不变。换言之,闭包的价值不在快、也不在好,而在知道哪些不必动——这正是一个「边界」应该提供的东西,也解释了为什么幅度与闭包排除掉多少成正比:在闭包接近全部未来预约的算例上(congested_8x4x4,释放占比 0.94)降幅只有 0.010,而在闭包排除得多的算例上(S8x4x4_mid,0.91)降幅达 0.118。

把这一正比写成一条可用的事后规则。图 8 以释放占比 $Cl/|R^0|$ 为横轴、RA 相对 R2 的改动比例降幅为纵轴,画出 50 对。在本样本上取 $Cl/|R^0| \geq 0.93$:该线以上 23 格平均降幅 0.029,以下 27 格平均降幅 0.094;占比再升到 0.97 的 8 格降幅为 0(闭包已是平凡上界,两臂恒等)。这是一条事后分档,阈值在本表上选定,不作外推定理;它回答的是「何时划这条边界还值」,不是闭包规模的结构预测(后者仍未建立,第 6.7 节)。

四臂并读的结论是:在「按 A2 可采纳、且响应在毫秒量级」这两个约束下,唯一比有界修复更快的做法(全局右移)在解质量与稳定性上都更差;受约束的重解码在完工时间上仍略优,但更慢、改动也更多;而在同等速度与同等解质量下,不划边界要多改约 6 个百分点的预约(相对多约一成二)。放宽响应时间约束,更好的完工时间仍能从热启动搜索买到(第 6.8 节),但那已不是毫秒档。

6.10 扰动规模扫描

表 11 与图 9 报告 φ 扫描(congested_8x4x4, 10 种子; R0+ 预算 2 s; STRC 含失败扩域)。两臂可行率均为 100%; STRC 耗时约 8–11 ms, R0+ 约 2 s; $C_{\max}$ 仍全由 R0+ 更优,与第 6.8 节一致。小例 example_3x3x2 上同样每格可行(10 种子 $\times$ 五个 φ 值),STRC 约 1 ms 量级,加速比 $87 \times - 3051 \times$ (跨两个算例的逐格极值)。

这里 STRC 打开了失败扩域,故与 E3/E5 不可直接比较:扩域是把可行率兜到 100% 的机制,而它本身并非本文首创(第 2.1 节)。本表的用途是说明,即使在扰动比例升至 $\varphi = 1$ (全部未来工序受扰)时,闭包路径仍停留在十余毫秒量级,响应时间不随扰动规模崩溃。

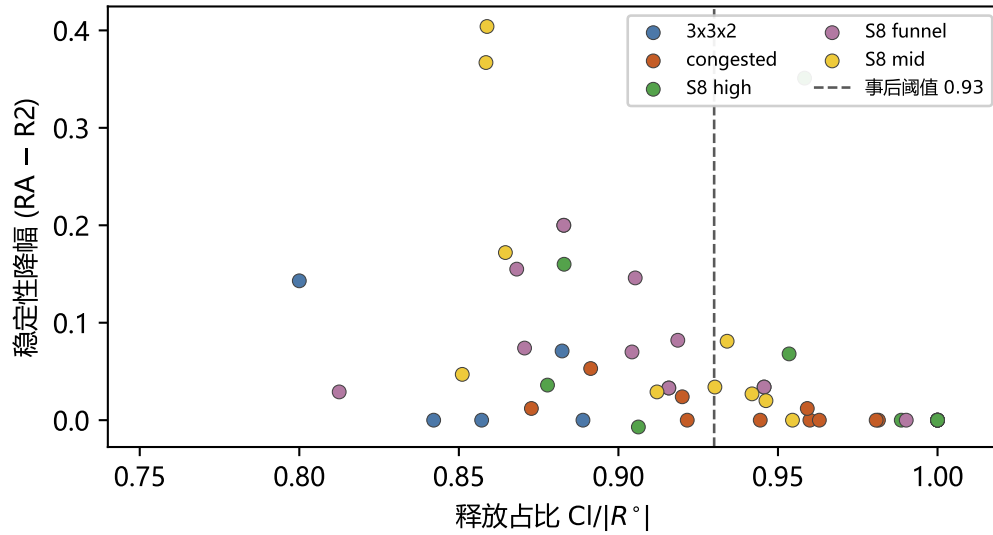


Fig. 8. E7 事后规则:释放占比与稳定性降幅(50 对, R2 对 RA)。虚线为样本内阈值 0.93;纵轴为正表示划边界后改动比例更低。

表 11. 扰动规模对照(congested_8x4x4, 10 种子平均)。STRC 含扩域;R0+ 预算 2s。

φ	STRC 可行	STRC ms	R0+ $C_{\max}$	耗时比
0.10	100%	9.1	103.5	406×
0.25	100%	11.1	107.0	325×
0.50	100%	8.6	111.8	333×
0.75	100%	8.4	125.8	254×
1.00	100%	10.7	148.1	203×

6.11 E8:算例规模上的外推

上一节扫的是扰动的规模,算例始终是 8 工件 4 机。本节换一维:固定扰动协议,把算例本身放大。这一问的要害不在耗时——耗时不崩是可以预期的——而在闭包占比会不会随规模涨到接近 1,使「有界」名存实亡。

梯子取工件数 $2k$ 、机器数 k 、车数 $k, k = 4, \dots, 10$, 即 $8 \times 4 \times 4$ 到 $20 \times 10 \times 10$, 共 7 档 $\times$ 10 种子 = 70 对。拥堵档、异构度、柔性度、运输/加工时长比与装卸点最小割、远端割全部按同一目标标定,只有规模变。三条臂与第 6.9 节同定义。

闭包占比不随规模增长。从 0.56 降到 0.48,趋势甚至是缓降的。这是本节最要紧的一条:有界性并未随规模被稀释。但须说明这不是一个渐近主张——七档不足以拟合任何增长率,能说的只是在这段区间内它没有退化。

响应时间随预约总数近似线性。 $|R|$ 增至 2.7 倍时, R2 的耗时中位从 2.9 增至 10.4 ms, 仍在十毫秒量级。第 1 级改路的可行格数为 69/70; 唯一失败的一格($10 \times 5 \times 5$, 种子 42)由失败扩域在第 3 轮兜住, 总耗时 12.0 ms、完工时间与首级尝试相同,故终可行为 70/70。

两个对照的差距随规模拉大,方向都对本文有利。最便宜的全局重算与有界修复的耗时比从 $1.5 \times$ 升到 $3.3 \times$ ——规模越大,重算一遍越不划算;而对不划边界的右移臂, R2 的 $C_{\max}$ 胜负从 7/1/2 变为最后三档的 10/0/0——规模越大,「只动该动的」相对「整体后推」的优势越明显。

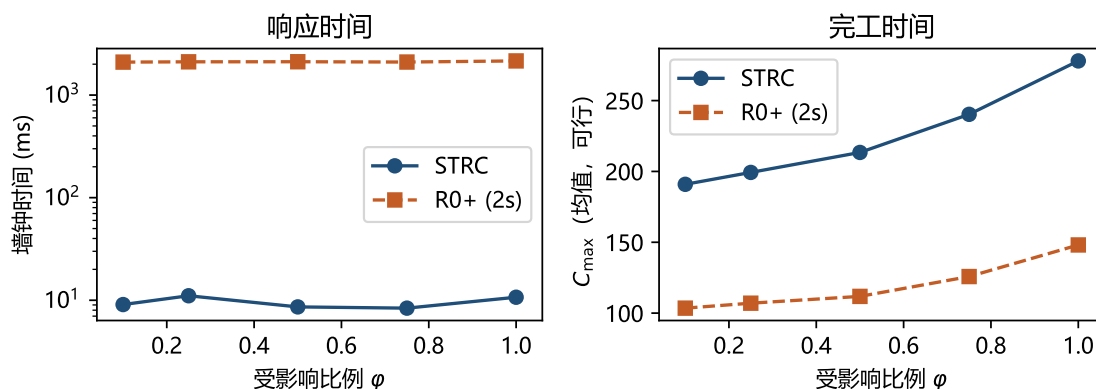


Fig. 9. 扰动规模扫描(congested_8x4x4, 10 种子): 响应时间(对数轴)与完工时间随 ϕ 变化。

表 12. E8 算例规模扫描(7 档 $\times$ 10 种子)。「闭包」为闭包预约占全表之比;耗时取中位。末列为 R2 对 RS 的 $C_{\max}$ 胜负。

规模	R	闭包	R2 ms	RD ms	RD/R2	R2 对 RS
$8 \times 4 \times 4$	148	0.561	2.9	4.4	1.5 $\times$	7/1/2
$10 \times 5 \times 5$	172	0.485	3.2	6.1	1.9 $\times$	9/0/1
$12 \times 6 \times 6$	230	0.481	4.5	10.7	2.4 $\times$	7/1/2
$14 \times 7 \times 7$	281	0.492	6.1	14.5	2.4 $\times$	8/0/2
$16 \times 8 \times 8$	291	0.534	6.4	16.4	2.6 $\times$	10/0/0
$18 \times 9 \times 9$	356	0.535	8.2	26.4	3.2 $\times$	10/0/0
$20 \times 10 \times 10$	402	0.476	10.4	34.2	3.3 $\times$	10/0/0

口径限制。这条梯子只有一个拥堵档、一个算例随机种子(每档的算例本身由种子 42 生成),因而它检验的是规模这一维,不能替代第 6.1 节主批次在拥堵结构上的对照。第 6.5 节的四类扰动交叉与第 6.10 节的扰动规模扫描也没有在这批算例上重跑。

6.12 外部布局批次的复现

第 6.1 节第二批把布局出处换成外部来源后,50 个算例-种子上 E1/E2/E3/E5 的读数与主批次逐项同向(E4 是唯一被改写的一格,见本节末;E5 与主批次的唯一差别是多出一个两臂持平的预算点,见下):任务图漏报 50/50(结构泄漏计 0),结构包含性 50/50,外侧字段逐条不变 50/50;边界消融下第 1 级改路的可行格数为 0/50,闭包边界为 50/50。E1 协议下的闭包占比逐布局中位落在 0.438–0.547,与主批次逐算例的 0.54–0.58 同量级。与全局重解的权衡亦无变化:17/18 个预算点上 R0+ 的 $C_{\max}$ 更优,余下 1 点两臂持平(LyuL6_8m,种子 2024,预算 0.2 s),没有一个点上有界修复更优,交叉点仍不存在,而有界修复的响应时间仍在毫秒量级、R0+ 仍在秒量级。

这一批的意义不在于给出新的性能数字,而在于把"这些现象是否只在本文自己设计的布局上成立"这一问的答案钉住:B 类扰动使任务图边界失效、预约闭包包含之、闭包外字段不受扰动,这三件事在一批不由本文选定的拓扑上同样成立。唯一被这批算例改写的结论是结构可预测性(第 6.7 节),而它本来就是负结果。

6.13 小结

- 问题层(E1)与边界消融(E3)在 50 个算例-种子上无一例外成立;案例甘特把同一协议落到工序时间轴。

- 边界覆盖矩阵(E6)在四类扰动 $\times 50$ 对上给出边界读数: A/B 二分无例外,且 A 类上闭包边界一致包含任务图边界;修复维四类都开口: B 类第 1 级均满(50/50/ 50/50),车辆故障换车 50/50, 机械臂故障改派 50/50。
- 结构包含性(E2a)与外侧字段逐条不变(E2b)均满通过;但这只说明命题 3 的四条前提在本文协议下成立,命题仍只作充分条件(注 4 记录了前提 (iii) 曾如何在实现上被悄悄破坏)。
- 与全局重解的关系是在响应时间约束下的分工,不是质量上的互有胜负: 毫秒 vs 秒、改动比例已与受 A2 约束的 R0+ 同在 50%–66% 量级,但 $C_{\max}$ 全区间劣势、无交叉。
- 便宜对照臂(E7)排除了「对照选贵了」这一疑问:在给出可行方案的各臂中,唯一比有界修复更快的是不划边界的全局右移,而它在 $C_{\max}$ (逐格 27/16/7)与稳定性 (0.627 对 0.540)上都输;而同一实现里最便宜的一次全局重算(原染色体重解码)比一次有界修复还贵 1.5 倍;RD 已走固定前缀,A2 越界 0/50。
- 同一批还把闭包单独隔离出来测了一次:换用「不划边界、释放全部未来」的对照臂(RA, 与 R2 共用引擎与冻结判据),耗时与 $C_{\max}$ 几乎不变(逐格 R2 赢/平/输 4/46/0), 改动比例却由 0.540 恶化到 0.604——逐格 31/18/1, $p = 8.8 \times 10^{-7}$ 。故毫秒级响应来自「单遍改路」而非闭包,闭包兑现的是稳定性, 且降幅与它排除掉多少成正比;样本内事后规则是释放占比超过 0.93 时平均降幅从 0.094 落到 0.029(图 8)。
- 算例规模扫描(E8)把梯子推到 20 工件 10 机:闭包占比从 0.56 缓降到 0.48(有界性未随规模稀释),响应时间随预约总数近似线性升至 10.4 ms,而与两个对照的差距同向拉大。这是一段区间内的读数,不是渐近主张。
- 闭包覆盖约 92% 的活预约,本文不作最小性主张。让行边纳入接壤后,差分试探在 50 对上泄漏为 0(0/50); 这只关闭了已观测到的那条通道。
- 三档非均匀边权上,任务图漏检与 R1 全败在 150 格上复现;第 1 级 R2 可行率在对数正态与距离梯度为 50/50/50/50,二分边权为 49/50。漏检命题对边权不敏感;可行率只在二分边上掉了一格。
- 结构可预测性(E4)是负结果,且换用外部来源布局后结论更硬:在同一张 4×4 外部路网上只改机器落位,闭包占比即变动 0.207(已是三张不同自建路网之间极差 0.051 的约四倍),而 LU 最小割与漏斗占比在整批外部算例上取值恒定,无从解释这一变动。故闭包规模不作可预测性主张,并据此把替代方向指向需求侧的时序密度。
- 两批算例承担不同的问题:主批次给出现象与机制,第二批只检验这些现象是否依赖本文自选的布局。把布局出处理换成外部来源后(50 对),E1/E2/E3/E5 的读数逐项复现, 无一条结论因换布局而改变(第 6.12 节)。第二批在全文中只用于收窄主张、未曾用于支持任何一条,且只有 E4 一格必须两批合读(第 6.1 节)。

7 结论

7.1 结论

本文指认了一类任务图覆盖不到的级联损坏,并将影响边界重新定义在时空预约闭包(STRC)上。在 5 算例 $\times 10$ 种子共 50 对上: 走廊阻断的任务图影响域 50/50 为空而原排程确已不可行; 同一修复引擎下仅替换边界定义时,任务图边界 0/50 恢复可行而闭包 50/50; 闭包的结构闭合性 50/50 成立,闭包外侧预约字段的逐条不变 50/50 成立; 四类扰动 $\times$ 两种边界的边界维覆盖矩阵上 A/B 二分无例外,且 A 类上闭包边界一致包含任务图边界。

主张的边界也已划清。让行关系这一建模对象、「局部重规划 + 外侧冻结」这一机制形态、以及「失败后扩大释放域」这一策略,分别在铁路 alternative graph、多智能体路径规划与 Local Rescheduling 中早已存在,本文不作创新声称(第 2.5 节)。本文主张的是三件事:该边界在两图分立的模型层级上被划错了; 改画之后它是一个由图结构唯一确定、有闭合性、可测量的对象而非启发式半径; 以及这个对象是一次测量而不是一个决策。

该对象买到什么,也已单独测过。把释放集从闭包换成「 t_{now} 之后的全部预约」而保持引擎、冻结判据与协议不变时,耗时与完工时间几乎不动,改动比例却由 0.540 恶化到 0.604(逐格 31/18/1, $p = 8.8 \times 10^{-7}$; 第 6.9 节)。故

毫秒级响应应归因于单遍改路这一机制形态——本文对它不作创新声称——而闭包兑现的恰是命题 3 承诺的那个量:外侧不动。这也界定了收益的形状:它与闭包排除掉多少成正比,因而在闭包接近平凡上界时趋于消失。

7.2 局限

以下四条按对结论的威胁程度排序,第 7.3 节逐条给出推进方向。

- (1) 完工时间上全区间劣于受 A2 约束的热启动全局重解,且不存在交叉点。在 18 个预算点上无一例外。对照臂已走固定前缀解码,历史不可改写审计为 0(第 6.8 节);E7 上 RD 越界同样为 0/50。差距比从 $t = 0$ 重排时小得多,但方向没变:本文买到的是响应时间,卖掉的仍是解质量。第 6.9 节已排除「对照选贵了」:同一实现里最便宜的一次全局重算比有界修复更慢,且已走固定前缀;与 RA 的对照则说明稳定性不来自这一臂。
- (2) 对闭包这个对象,本文知道的仍不够:保证不充要、边集未经独立检验、释放集不紧、规模也不可由静态结构预测。四者分述。
 - (i) 命题 3 只是充分条件,50/50 不等于充要:实验说明的是四条前提在本文协议下成立,不是它们可以省略。前提 (iii) 尤其脆弱——只要实现以比预约更粗的粒度决定重规划范围, U 外的占用就会被顺带改写(注 4);本文没有给出一个能自动检出该类粒度错配的手段。更需注意的是,逐条比对外侧字段(E2b)不足以兜住这类错配:同一注记的后半段记录了一次只发生在闭包内部、因而 E2b 完全看不见的历史改写,它要靠一项独立的历史不可改写审计才暴露出来。因此本文对该类缺陷的检出依赖于「事后想到该查什么」,而非一个完备的检查。
 - (ii) 依赖边集并不穷尽物理耦合,但已观测到的那条接壤通道已经关掉。E2b 满通过检验的是修复实现与释放集定义是否一致,不是四类边覆盖了一切传播通道。差分试探——只把闭包内一条预约后移一个单位,观察闭包外是否出现新的同走廊重叠——在把接壤并入让行边并重跑主矩阵之后,50 对上泄漏为 0(0/50)。同车方向始终为 0。这不是完备性证明:试探只覆盖「延后 1 个时间单位」这一种扰动,其它耦合通道仍未检验(注 2)。
 - (iii) 不作最小性主张,且闭包确实不小:它覆盖 t_{now} 之后活预约的约 92%(注 2);相对「释放全部未来」这一平凡上界,结构化释放省下的不足一成。这一条的代价已实测(第 6.9 节的 RA 臂):那不足一成的差别换来改动比例降 0.064(逐格 31/18/1, $p = 8.8 \times 10^{-7}$),而耗时与 C_{max} 不变。所以限制不在于「闭包无用」,而在于收益幅度不大且与闭包排除掉多少成正比——在闭包接近全部未来预约的算例上降幅仅 0.010。事后规则(图 8)把「何时」落成一条样本内分档: $CI/|R^c| \geq 0.93$ 时平均降幅 0.029(23 格),低于该线为 0.094(27 格);它不是结构预测,也不能外推。
 - (iv) 闭包规模的结构可预测性未能建立,且已知不由装卸点处的静态割给出:最小割与漏斗占比都没有做出可复现的区分;换用外部来源布局后这一点变得更确定——这两个指标在整批外部算例上取值恒定(两张路网都把装卸站放在网格角点,四邻接下角点恰有两条出边),而闭包占比在同一张路网上仅换机器落位就变动 0.207(第 6.7 节)。须注意这条否证的适用范围:外部算例在路网几何这一维只有 2 个独立样本,故本文否证的是「静态割能解释闭包规模」,不是「路网几何与闭包规模无关」——后者需要更多张互不相同的公开路网才能检验,而可得的路网就这么多。下一个候选刻画应是 t_{now} 邻域内走廊占用的时序密度,本文未做。
- (3) 实现覆盖仍窄于框架所述:修复维已满、一档半恢复动作、一次响应。扰动一维,四类都有注入通道且修复均满(50/50/ 50/50/ 50/50/ 50/50,第 6.5 节)。动作一维,B 类仍只改路;A 类开口换车与改派,仍不改扫描序。放开改序即向全局重解退化,「有界」也随之失去边界。时间一维,由假设 A3,扰动在 t_{now} 完全已知且不叠加;扰动流、预测误差与滚动响应均未涉及。规模一维的情况好一些但仍有限:第 6.11 节把梯子推到 20 工件 10 机、70 对,闭包占比未随规模上升;但那条梯子只有一个拥堵档与一个算例种子,7 档也不足以支撑任何渐近论断,更大规模与更多结构下的行为仍未知。

(4) 外部参照缺位:这一支没有可换的公开基准,算例出处只外部化了一半,结论亦未在第二套实现上复现。带无冲突运输的柔性作业车间已有若干公开算例族,但执行期扰动这一支没有:公开族发布的都是静态问题的输入。更硬的一层障碍在算例格式——主流族把运输时间发布为机器间 **行程时间矩阵**而不是路网,我们对两族共 11 张布局逐张检验过 [4, 7],没有一张能还原成与所发布矩阵一致的无向走廊图(全部不对称,即同一对位置往返时长不等;其中两张连有向图也还原不出,因为违反三角不等式)。矩阵里没有走廊,就没有走廊占用,于是 R 、 G_R 、 $CI(\cdot)$ 与 B 类扰动在这类算例上一概无从定义。这比静态工作受到的约束更紧:静态工作可以把争用约束退化掉,退化后求解的仍是文献求解的那个问题,因而仍能借公开算例标定搜索能力;本文没有这条退路——退化掉争用,被研究的对象本身就不存在了。唯一发布显式双向排他路网的是 Lyu 等 [12]。本文已按其附录布局做出第二批算例(第 6.1 节),布局因此不再由本文设计;但外部化只做到拓扑一层——逐段行驶时间与工件数据仍是本文的。前者原文未随该批布局发表(只随单个示例算例给出,且取值并不均匀),后者若一并借用,公开族过低的运输/加工时长比会使争用现象直接消失。所以这批算例不能用来复现 Lyu 或 van Os [22] 的参照值;后者的模型另有「一个节点同时只容一车」的约束,与本文的走廊排他语义也不是同一回事。算例出处这一条因此只解决了一半:布局那一半解决了,时间标定那一半没有。等权之外另扫了三档非均匀边权(对数正态、离装卸点越远越慢、随机 0.5/2),只重跑 E1/E3: 任务图漏检与 R1 全败在 150 格上复现;R2 第 1 级可行率分别为 50/50、50/50、49/50。漏检命题对边权不敏感;第 1 级可行率只在二分边上掉了一格(49/50),距离梯度在补上接壤边后回到 50/50。这不能替代一份同时发布路网与逐段时长的外部算例族。外部批次的覆盖范围也窄于主批次:它复现的是 E1/E2/E3/E5(第 6.12 节),E6 的四类扰动交叉、扰动规模扫描以及第 6.9 与 6.11 节的两批(便宜对照臂、算例规模)都没有在外部布局上重跑;这四项的读数目前只有自建算例支撑。此外,该批的 5 个算例只落在 2 张路网上(第 6.1 节),路网几何这一维的样本量因此很小。最后,下层路网、时间窗路由与校验器与静态一体化工作复用同一实现:这保证了归因的干净,但也意味着上述结论尚未在第二套独立实现上复现。

7.3 后续工作

下列四件与上一节四条局限一一对应。

一、正视完工时间上的全区间劣势(局限 1)。前缀解码的历史空载泄漏已在主批次收干净(E5 的 18 个预算点越界 0, E7 上 RD 为 0/50)。局限 1 剩下的就是 $C_{\max}$ 本身:不打算换加权指标绕过去。若要缩小差距,需要的是受 A2 约束的有限度改派或改序,那会向全局重解退化,而这正是本文不把它们当作创新的原因。

二、把闭包的保证做实、做紧,并给它的规模找一个刻画(局限 2)。做实:接壤已并入让行边,主矩阵重跑后 E1/E2/E3 仍满通过,试探泄漏为 0。下一步是换一种扰动形态(例如缩短占用、或延后超过 1 个单位)再探,而不是再补同一条边。做紧,指向最小释放集逼近:在固定修复能力下定义最小性,并给出闭包与之的差距,这是把「有界」从充分条件推进到紧界的必经一步;当前闭包覆盖约 92% 的活预约,差距很可能不小。刻画规模已有一条事后规则(释放占比,图 8),但它随排程而变;要换一族可在扰动前计算的指标:第 6.7 节已把静态割这一族排除——它在真实布局上取值恒定,而同一张路网上仅改机器落位,闭包占比就变动 0.207。后者正好提示了方向:起作用的似乎是需求落在哪里而不是路网提供了什么。下一个候选是 t_{now} 邻域内的走廊占用时序密度,例如受扰走廊在其繁忙窗口内的重叠预约条数、以及该走廊在让行图上的出度。这类量随排程而变,不再是纯拓扑量,因而也需要重新界定「可预测性」问的是什么:是给定布局预测闭包规模,还是给定布局与当前排程预测它。

三、放开 A3(局限 3;假设 A3 为单次、完全观测)。覆盖矩阵修复四维均已满:阻断与降速第 1 级、车辆故障换车、机械臂故障改派。下一步不是再开改序,而是放开该条,考察闭包在连续扰动下是否会累积膨胀,以及两次修复之间是否需要显式的「归位」步骤。

四、把时间标定也外部化(局限 4)。布局出处已解决一半;等权之外的三档边权说明漏检命题站住了,第 1 级可行率没有。要补齐另一半,仍然需要一份同时发布 排他路网与逐段时长的算例族——自建边权扫不能替代它。至于第二套独立实现,它不必单独立项——上述任何一件由他人复现时都会顺带完成。

References

- [1] Riyad J. Abumaizar and Joseph A. Svestka. 1997. Rescheduling job shops under random disruptions. *International Journal of Production Research* 35, 7 (1997), 2065–2082. doi:10.1080/002075497195074
- [2] M. Selim Aktürk and Elif Görgülü. 1999. Match-up scheduling under a machine breakdown. *European Journal of Operational Research* 112, 1 (1999), 81–97. doi:10.1016/S0377-2217(97)00396-2
- [3] James C. Bean, John R. Birge, John Mittenhal, and Charles E. Noon. 1991. Matchup scheduling with multiple resources, release dates and disruptions. *Operations Research* 39, 3 (1991), 470–483. doi:10.1287/opre.39.3.470
- [4] Ümit Bilge and Gündüz Ulusoy. 1995. A Time Window Approach to Simultaneous Scheduling of Machines and Material Handling System in an FMS. *Operations Research* 43, 6 (1995), 1058–1070. doi:10.1287/opre.43.6.1058
- [5] Andrea D’Ariano, Dario Pacciarelli, and Marco Pranzo. 2007. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research* 183, 2 (2007), 643–657. doi:10.1016/j.ejor.2006.10.034
- [6] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production–Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.
- [7] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6
- [8] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139
- [9] Jürgen Kuster, Dietmar Jannach, and Gerhard Friedrich. 2007. Local Rescheduling: A novel approach for efficient response to schedule disruptions. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched)*. IEEE, 79–86. doi:10.1109/SCIS.2007.367673
- [10] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543
- [11] Rong-Kwei Li, Yu-Tang Shyu, and Sadashiv Adiga. 1993. A heuristic rescheduling algorithm for computer-based production scheduling systems. *International Journal of Production Research* 31, 8 (1993), 1815–1826. doi:10.1080/00207549308956824
- [12] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109
- [13] Nir Malka, Guy Shani, and Roni Stern. 2024. Online Planning for Multi Agent Path Finding in Inaccurate Maps. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 10214–10221. doi:10.1109/IROS58592.2024.10801975
- [14] Alessandro Mascis and Dario Pacciarelli. 2002. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research* 143, 3 (2002), 498–517. doi:10.1016/S0377-2217(01)00338-1
- [15] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355
- [16] Djamilia Ouelhadj and Sanja Petrovic. 2009. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* 12, 4 (2009), 417–431. doi:10.1007/s10951-008-0090-8
- [17] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352
- [18] Velusamy Subramaniam and Amritpal Singh Raheja. 2003. mAOR: A heuristic-based reactive repair mechanism for job shop schedules. *The International Journal of Advanced Manufacturing Technology* 22, 9-10 (2003), 669–680. doi:10.1007/s00170-003-1601-6
- [19] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526
- [20] Jiří Svancara, Marek Vlček, Roni Stern, Dor Atzmon, and Roman Barták. 2019. Online Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 7732–7739. doi:10.1609/aaai.v33i01.33017732
- [21] Hongtao Tang, Xuewei Zhang, Jiawei Huang, Xuesong Xu, and Jiansha Lu. 2026. Integrated Scheduling of Multi-Objective Lot-Streaming Hybrid Flowshop with AGV under Dynamic Environments. *International Journal of Industrial Engineering Computations* 17, 2 (2026), 295–316. doi:10.5267/j.ijiec.2025.9.003
- [22] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

- [23] Guilherme E Vieira, Jeffrey W Herrmann, and Edward Lin. 2003. Rescheduling Manufacturing Systems: A Framework of Strategies, Policies, and Methods. *Journal of Scheduling* 6, 1 (2003), 39–62. doi:10.1023/A:1022235519958
- [24] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795
- [25] Yannik Zeiträg and José Rui Figueira. 2025. A novel machine-learning rolling horizon heuristic for dynamic lot-sizing and job shop scheduling problems. *International Journal of Production Research* 63, 12 (2025), 4563–4589.
- [26] Hongliang Zhang, Chaoqun Qin, Wenhui Zhang, Zhenxing Xu, Gongjie Xu, and Zhenhua Gao. 2023. Energy-Saving Scheduling for Flexible Job Shop Problem with AGV Transportation Considering Emergencies. *Systems* 11, 2 (2023), 103. doi:10.3390/systems11020103
- [27] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*.
- [28] Ziyang Zhao, Shixin Liu, MengChu Zhou, Xiwang Guo, and Liang Qi. 2019. Decomposition method for new single-machine scheduling problems from steel production systems. *IEEE Transactions on Automation Science and Engineering* 17, 3 (2019), 1376–1387.